

**ĐẠI HỌC CẦN THƠ
TRƯỜNG CÔNG NGHỆ THÔNG TIN & TRUYỀN THÔNG**



**LUẬN VĂN TỐT NGHIỆP
NGÀNH KHOA HỌC MÁY TÍNH**

Đề tài

**XÂY DỰNG TRỢ LÝ ẢO
HỖ TRỢ HOẠT ĐỘNG TIẾP XÚC CỬ TRI**

**DEVELOPING A VIRTUAL ASSISTANT
TO SUPPORT VOTER ENGAGEMENT MEETINGS**

Sinh viên: Lê Tuấn Đạt

Mã số: B2113328

Khóa: 47

Giảng viên hướng dẫn: TS. Trần Nguyễn Minh Thư

Cần Thơ, 07/2025

**ĐẠI HỌC CẦN THƠ
TRƯỜNG CÔNG NGHỆ THÔNG TIN & TRUYỀN THÔNG**



**LUẬN VĂN TỐT NGHIỆP
NGÀNH KHOA HỌC MÁY TÍNH**

Đề tài

**XÂY DỰNG TRỢ LÝ ẢO
HỖ TRỢ HOẠT ĐỘNG TIẾP XÚC CỬ TRI**

**DEVELOPING A VIRTUAL ASSISTANT
TO SUPPORT VOTER ENGAGEMENT MEETINGS**

**Giảng viên hướng dẫn
TS. Trần Nguyễn Minh Thư**

**Sinh viên thực hiện
Họ và tên: Lê Tuấn Đạt
Mã số: B2113328
Khóa: 47**

Cần Thơ, 07/2025

NHẬN XÉT CỦA GIÁO VIÊN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Cần Thơ, ngày 19 tháng 08 năm 2025

TS. Trần Nguyễn Minh Thư

LỜI CẢM ƠN

Để có được bài báo cáo này, em xin được bày tỏ lòng biết ơn chân thành và sâu sắc đến Cô Trần Nguyễn Minh Thư – người đã trực tiếp tận tình hướng dẫn, giúp đỡ em. Trong suốt quá trình thực hiện niên luận, nhờ những sự chỉ bảo và hướng dẫn quý giá đó mà bài niên luận này được hoàn thành một cách tốt nhất.

Em cũng xin gửi lời cảm ơn chân thành đến các Thầy Cô Giảng viên Đại học Cần Thơ, đặc biệt là các Thầy Cô ở Trường CNTT & TT, những người đã truyền đạt những kiến thức quý báu trong thời gian qua.

Em cũng xin chân thành cảm ơn bạn bè cùng với gia đình đã luôn động viên, khích lệ và tạo điều kiện giúp đỡ trong suốt quá trình thực hiện để em có thể hoàn thành bài niên luận một cách tốt nhất.

Tuy có nhiều cố gắng trong quá trình thực hiện niên luận, nhưng không thể tránh khỏi những sai sót. Em rất mong nhận được sự đóng góp ý kiến quý báu của quý Thầy Cô và các bạn để bài niên luận hoàn thiện hơn.

Cần Thơ, ngày 19 tháng 08 năm 2025

Người viết

(Ký và ghi họ tên)

Lê Tuấn Đạt

MỤC LỤC

LỜI CẢM ƠN	ii
MỤC LỤC	iv
DANH MỤC HÌNH ẢNH	i
DANH MỤC BẢNG BIỂU	ii
DANH MỤC TỪ VIẾT TẮT	iii
ABSTRACT	iv
TÓM TẮT	v
I. PHẦN GIỚI THIỆU	1
1. Đặt vấn đề	1
2. Lịch sử giải quyết vấn đề	2
2.1. Truy xuất thông tin dựa trên từ khóa	2
2.2. Học máy cổ điển và trích xuất thông tin:	3
2.3. Học sâu và các mô hình ngôn ngữ ngữ cảnh:	3
2.4. Thế hệ mới - Truy xuất tăng cường cho sinh ngôn ngữ (RAG)	4
3. Mục tiêu đề tài	5
4. Đối tượng và phạm vi nghiên cứu	6
4.1. Đối tượng nghiên cứu:	6
4.2. Phạm vi nghiên cứu:	6
5. Phương pháp nghiên cứu	6
6. Kết quả đạt được:	7
7. Bố cục quyền báo cáo	7
II. PHẦN NỘI DUNG	8
CHƯƠNG 1. MÔ TẢ BÀI TOÁN	8
1.1. Mô tả chi tiết bài toán	8
1.2. Vấn đề và giải pháp liên quan đến bài toán	10
1.2.1. Kiến trúc Transformer:	10
1.2.2. Mô hình ngôn ngữ lớn (Large Language Models - LLMs):	11
1.2.3. Hệ thống hỏi đáp (Question Answering - QA):	13
1.2.4. Retrieval-augmented Generation (RAG):	14
1.2.5. Nhúng vector (Vector embedding):	16
1.2.6. Cơ sở dữ liệu vector (Vector database/stores):	16
1.2.7. Phương pháp đánh giá hệ thống - RAGAS:	17
1.2.8. Các thư viện và công cụ hỗ trợ:	17

1.2.8.1. Requests:	17
1.2.8.2. Selenium:	18
1.2.8.3. BeautifulSoup:	18
1.2.8.4. PyMuPDF:	18
1.2.8.5. LangChain:	18
1.2.8.6. RAGAS:	19
1.2.8.7. Flask:	19
CHƯƠNG 2. THIẾT KẾ VÀ CÀI ĐẶT GIẢI PHÁP	20
2.1. Thiết kế hệ thống	20
2.2. Cài đặt giải pháp	21
2.2.1. Thu thập dữ liệu	21
2.2.2. Tiền xử lý dữ liệu	27
2.2.2.1. Trích xuất văn bản thô	27
2.2.2.2. Làm sạch văn bản	28
2.2.2.3. Bóc tách siêu dữ liệu (metadata):	31
2.2.2.4. Chia đoạn văn bản	32
2.2.2.5. Chuyển đổi cây cấu trúc thành các chunk ngữ nghĩa:	34
2.2.2.6. Nhúng văn bản và lưu trữ vào ChromaDB	36
2.2.3. Xử lý truy vấn và sinh câu trả lời	37
2.2.3.1. Tiền xử lý và nhúng câu hỏi	37
2.2.3.2. Tìm kiếm ngữ cảnh (Retrieval):	38
2.2.3.3. Sinh câu trả lời và hiển thị kết quả:	38
CHƯƠNG 3. KIỂM THỬ VÀ ĐÁNH GIÁ	40
3.1. Giao diện sản phẩm	40
3.1.1. Thanh điều khiển - Quản lý phiên (Sidebar):	40
3.1.2. Khu vực hội thoại chính (Main Chat Area):	41
3.2. Kết quả thực nghiệm	41
3.2.1. Thống kê dữ liệu qua các bước tiền xử lý:	41
3.2.2. Thiết lập môi trường và dữ liệu thực nghiệm:	42
3.2.3. Kết quả đánh giá định lượng:	43
3.2.4. Kết quả đánh giá định tính:	44
III. PHẦN KẾT LUẬN	46
1. Kết quả đạt được	46
2. Hướng phát triển	46
TÀI LIỆU THAM KHẢO	48

DANH MỤC HÌNH ẢNH

Hình 1. Quá trình tiếp xúc cử tri (Nguồn: baochinhpvu.vn)	1
Hình 2. Giao diện tìm kiếm văn bản pháp luật với bộ lọc từ khóa và thông tin hành chính (Nguồn: vbpl.vn)	2
Hình 3. Sơ đồ kiến trúc Transformer (Nguồn: Vaswani et al., 2017)	3
Hình 4. Sơ đồ kỹ thuật RAG	4
Hình 5. Giao diện ứng dụng tra cứu, hỏi đáp pháp luật của dự án VN-Law-Advisor	5
Hình 6. Ví dụ về một văn bản quy phạm pháp luật đầu vào cho hệ thống địa điểm tập trung đông người dịp Tết Dương lịch và Tết Nguyên Đán năm 2025)	9
Hình 7. Kiến trúc Transformer (Nguồn: Vaswani et al., 2017)	11
Hình 8. Kiến trúc kỹ thuật RAG (Nguồn: Lewis et al., 2020)	14
Hình 9. Quy trình tổng thể của hệ thống	20
Hình 10. Trang Cổng Thông tin điện tử Chính phủ (Nguồn: https://congbao.chinhphu.vn/)	21
Hình 11. Quy trình cào dữ liệu	23
Hình 12. Cấu trúc HTML của trang chứa loại văn bản “Chỉ thị”	24
Hình 13. Mã nguồn trang chi tiết chứa đường dẫn tải văn bản trong thẻ 	26
Hình 14. Minh họa kết quả của bước trích xuất chi tiết	28
Hình 15. Hình 14a và 14b là các thông tin gây nhiễu, không cần lưu trữ	29
Hình 16. Các trường và nội dung của các siêu dữ liệu sau khi bóc tách metadata	32
Hình 17. Mục 1, khoản a, b của chỉ thị 28/CT-TTg năm 2015	32
Hình 18. Giao diện hệ thống	40
Hình 19. Kết quả trường hợp 1	44
Hình 20. Kết quả trường hợp 2	44
Hình 21. Kết quả trường hợp 3	45

DANH MỤC BẢNG BIỂU

Bảng 1. Các thành phần cốt lõi của hệ thống RAG	15
Bảng 2. Bảng thống kê quá trình xử lý dữ liệu	41
Bảng 3. Phân loại và ví dụ các câu hỏi trong bộ dữ liệu đánh giá	43
Bảng 4. Kết quả đánh giá hiệu suất hệ thống bằng RAGAS	43

DANH MỤC TỪ VIẾT TẮT

STT	Từ viết tắt (Abbreviation)	Từ viết đầy đủ (Origin word)
1	VPPL	Văn bản quy phạm pháp luật
1	LLMs	Large Language Models
2	QA	Question Answering
3	NLP	Natural Language Processing
4	GPT	Generative Pre-trained Transformer
5	BERT	Bidirectional Encoder Representations from Transformers
6	T5	Text-To-Text Transfer Transformer
7	PDF	Portable Document Format
8	SVM	Support Vector Machine
9	CRF	Conditional Random Fields
10	RNN	Recurrent Neural Network
11	LSTM	Long-Short Term Memory
12	TF-IDF	Term Frequency-Inverse Document Frequency
13	BiDAF	Bi-Directional Attention Flow
14	DrQA	Document Reader for Question Answering
15	ViT5	Vietnamese Text-To-Text Transfer Transformer

ABSTRACT

The thesis "Developing a Virtual Assistant to Support Voter Engagement Meetings" addresses the challenge officials and representatives face in quickly and accurately retrieving information from legal documents during public consultations. To tackle this, a question-answering system was developed based on the advanced Retrieval-Augmented Generation (RAG) architecture. The solution features an automated data preprocessing pipeline that extracts text from PDF files using PyMuPDF and employs semantic chunking to preserve the logical structure of the documents. The system integrates a hybrid retrieval mechanism, combining semantic search with BM25 keyword search, utilizing the text-embedding-004 model and a ChromaDB vector database. The Gemini 2.5 Pro large language model is used to synthesize information and generate natural language responses. Experimentally, the system demonstrated high performance, achieving a Faithfulness score of 0.9833 and a Context Precision score of 1.0000 on the RAGAS framework, confirming its reliability and effectiveness in mitigating hallucinations. A complete web application was also built using Flask, providing an intuitive interface for users to interact with the assistant and manage chat sessions.

Keywords: Legal Question Answering, Retrieval-Augmented Generation (RAG), Virtual Assistant.

TÓM TẮT

Luận văn "Xây dựng trợ lý ảo hỗ trợ hoạt động tiếp xúc cử tri" giải quyết thách thức trong việc tra cứu văn bản quy phạm pháp luật một cách nhanh chóng và chính xác cho cán bộ, đại biểu. Để giải quyết vấn đề này, một hệ thống hỏi-đáp đã được phát triển dựa trên kiến trúc tiên tiến Retrieval-Augmented Generation (RAG). Giải pháp bao gồm một pipeline tiền xử lý dữ liệu tự động, sử dụng PyMuPDF để trích xuất nội dung từ tệp PDF, sau đó thực hiện phân đoạn theo ngữ nghĩa để bảo toàn cấu trúc logic của văn bản. Hệ thống kết hợp cơ chế truy xuất hybrid (tìm kiếm ngữ nghĩa và từ khóa BM25), sử dụng mô hình nhúng text-embedding-004 và lưu trữ trên cơ sở dữ liệu vector ChromaDB. Mô hình ngôn ngữ lớn Gemini 2.5 Pro được tích hợp để tổng hợp thông tin và sinh câu trả lời tự nhiên. Về mặt thực nghiệm, hệ thống đạt hiệu suất cao với điểm Faithfulness là 0.9833 và Context Precision là 1.0000 qua bộ đánh giá RAGAS, chứng tỏ khả năng trả lời đáng tin cậy và giảm thiểu hiệu quả hiện tượng "ảo giác". Về sản phẩm, một ứng dụng web hoàn chỉnh được xây dựng bằng Flask, cung cấp giao diện trực quan cho phép người dùng tương tác và quản lý các phiên hội thoại.

Từ khóa: Hỏi đáp pháp luật, Trợ lý ảo, Truy xuất tăng cường cho sinh ngôn ngữ (RAG).

I. PHẦN GIỚI THIỆU

1. Đặt vấn đề

Trong bối cảnh dữ liệu văn bản ngày càng gia tăng về mặt số lượng và độ phức tạp, việc truy xuất thông tin một cách hiệu quả từ các tài liệu như văn bản quy phạm pháp luật (VPPL) trở thành một yêu cầu thiết yếu. Các văn bản như nghị định, nghị quyết, thông tư,... thường mang tính chất kỹ thuật, có độ dài lớn, cấu trúc phức tạp, và đòi hỏi độ chính xác cao trong quá trình tiếp cận và trích xuất thông tin. Điều này khiến cho không chỉ người dân mà cả cán bộ hành chính cũng gặp nhiều khó khăn trong việc tra cứu, hiểu và sử dụng nội dung các văn bản này. Việc ghi nhớ toàn bộ nội dung, hoặc nhanh chóng tìm ra phần thông tin phù hợp khi cần thiết, vẫn là thách thức lớn trong thực tiễn áp dụng.

Khó khăn này càng thể hiện rõ hơn trong bối cảnh tiếp xúc cử tri - một hoạt động chính trị định kỳ, nơi đại biểu Quốc hội hoặc Hội đồng Nhân dân (HĐND) tiếp nhận ý kiến, phản ánh và kiến nghị từ người dân. Cử tri thường đặt ra nhiều câu hỏi liên quan đến các quy định pháp luật, đặc biệt xoay quanh những vấn đề thời sự có sự ảnh hưởng sâu rộng, như việc sáp nhập đơn vị hành chính. Hiện tại, đây là chủ đề nóng, thu hút nhiều sự quan tâm, kéo theo hàng loạt câu hỏi như: “Số đồ có phải làm lại không?”, “Tôi đang hưởng chính sách ở phường cũ thì sau sáp nhập thế nào?”, “Có cần phải đi làm giấy tờ lại hay không?”. Những câu hỏi này đòi hỏi phản hồi chính xác, căn cứ vào văn bản pháp luật cụ thể. Tuy nhiên, không phải lúc nào đại biểu hay cán bộ hỗ trợ cũng có đủ thời gian và công cụ để nhanh chóng tra cứu, đối chiếu, và trích dẫn chính xác các điều khoản pháp lý liên quan. Điều này làm cho người hỗ trợ không đáp ứng ngay tức thì câu hỏi của người dân vô tình chung làm cho buổi tiếp xúc không được hiệu quả.



Hình 1. Quá trình tiếp xúc cử tri (Nguồn: baohinhphu.vn)

Song song đó, sự phát triển vượt bậc của lĩnh vực xử lý ngôn ngữ tự nhiên (NLP) đã mở ra những hướng đi mới. Các công trình tiên phong như “Attention is All You Need” của Vaswani và cộng sự (2017) đã giới thiệu kiến trúc Transformer, tạo nền tảng cho một thể hệ mô hình ngôn ngữ lớn (LLMs). Đặc biệt, kỹ thuật “Retrieval-Augmented Generation” (RAG), được giới thiệu trong công trình của Lewis và các cộng sự vào năm 2020, đã giải quyết bài

toán tạo sinh thông tin dựa trên một tri thức bên ngoài. Hướng tiếp cận này giúp nâng cao đáng kể độ chính xác và cho phép hệ thống trích dẫn nguồn gốc của câu trả lời, giải quyết những hạn chế cốt lõi của các mô hình ngôn ngữ lớn trước đó.

Từ thực tiễn đó, đề tài “Xây dựng trợ lý ảo hỗ trợ hoạt động tiếp xúc cử tri” ra đời với mục tiêu ứng dụng các kỹ thuật mới nhất trong lĩnh vực xử lý ngôn ngữ tự nhiên (NLP - Natural Language Processing), nhằm hỗ trợ người dùng - đặc biệt là đại biểu, cán bộ hoặc người dân - có thể truy xuất thông tin pháp luật một cách thông minh, chính xác và thuận tiện, phục vụ tốt hơn cho công tác đối thoại chính sách, tiếp xúc cử tri và áp dụng pháp luật vào thực tiễn.

2. Lịch sử giải quyết vấn đề

Việc tra cứu, trích xuất và hỏi đáp thông tin từ văn bản pháp luật luôn là một thách thức lớn, đặc biệt đối với đại biểu Quốc hội, Hội đồng Nhân dân và cán bộ hỗ trợ trong quá trình tiếp xúc cử tri. Các văn bản pháp luật thường có độ dài lớn, cấu trúc phức tạp và ngôn ngữ chuyên ngành, khiến việc tìm kiếm, đối chiếu chính xác các điều khoản liên quan để trả lời câu hỏi của người dân trở nên khó khăn và tốn nhiều thời gian. Trong thực tế, đại biểu và cán bộ thường phải đọc kỹ từng văn bản dài để tìm ra thông tin phù hợp, điều này ảnh hưởng trực tiếp đến hiệu quả và độ chính xác của các buổi tiếp xúc.

2.1. Truy xuất thông tin dựa trên từ khóa

Đây là hướng đi sơ khởi và phổ biến nhất, dựa trên các hệ thống truy xuất thông tin (Information Retrieval - IR) truyền thống. Nguyên lý hoạt động của các hệ thống này là lập chỉ mục (indexing) toàn bộ kho văn bản và sử dụng các thuật toán như TF-IDF để tìm kiếm sự trùng khớp của từ khóa do người dùng cung cấp. Tại Việt Nam, các cổng thông tin pháp luật lớn như Cơ sở dữ liệu quốc gia về văn bản pháp luật (vbpl.vn), Thư Viện Pháp Luật hay Luật Việt Nam là những ví dụ điển hình cho hướng tiếp cận này.



Hình 2. Giao diện tìm kiếm văn bản pháp luật với bộ lọc từ khóa và thông tin hành chính (Nguồn: vbpl.vn)

Tuy nhiên, hạn chế cố hữu của phương pháp này là không có khả năng “hiểu” được ngữ nghĩa hay ý định của người dùng. Hệ thống chỉ trả về các văn bản có từ khóa, và người

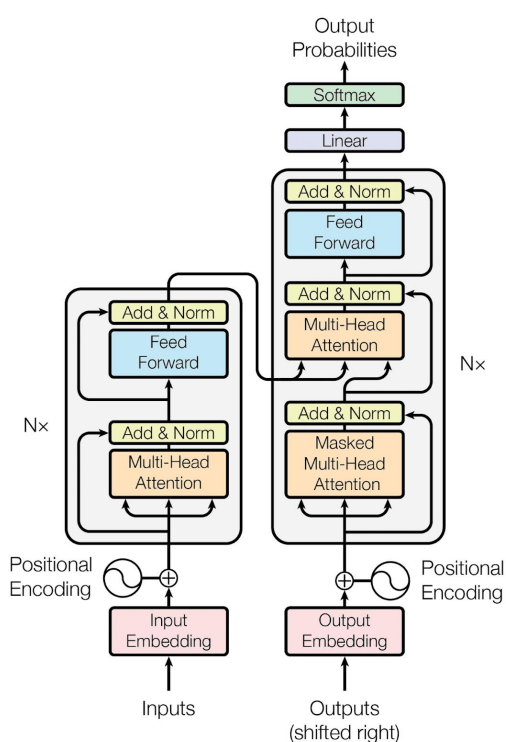
dùng cuối - dù là cán bộ hay người dân - vẫn phải tự đọc và phân tích để tìm ra câu trả lời chính xác, gây mất nhiều thời gian và có thể dẫn đến sai sót.

2.2. Học máy cổ điển và trích xuất thông tin:

Để khắc phục những hạn chế trên, các nhà nghiên cứu bắt đầu áp dụng những mô hình học máy cổ điển. Các phương pháp dựa trên luật định (rule-based) sử dụng biểu thức chính quy để nhận diện thông tin ban đầu. Tiếp đó, các mô hình như Naive Bayes, Support Vector Machine (SVM) và Conditional Random Fields (CRF) được ứng dụng để tự động phân loại văn bản theo chủ đề hoặc trích xuất các thực thể có cấu trúc như tên điều khoản, ngày ban hành, v.v.. Mục tiêu của hướng đi này là biến các văn bản phi cấu trúc thành dữ liệu có cấu trúc, từ đó hỗ trợ việc xây dựng đặc trưng thủ công, đồng thời gặp khó khăn trong việc nắm bắt các mối quan hệ ngữ nghĩa phức tạp trong các văn bản pháp luật dài.

2.3. Học sâu và các mô hình ngôn ngữ ngữ cảnh:

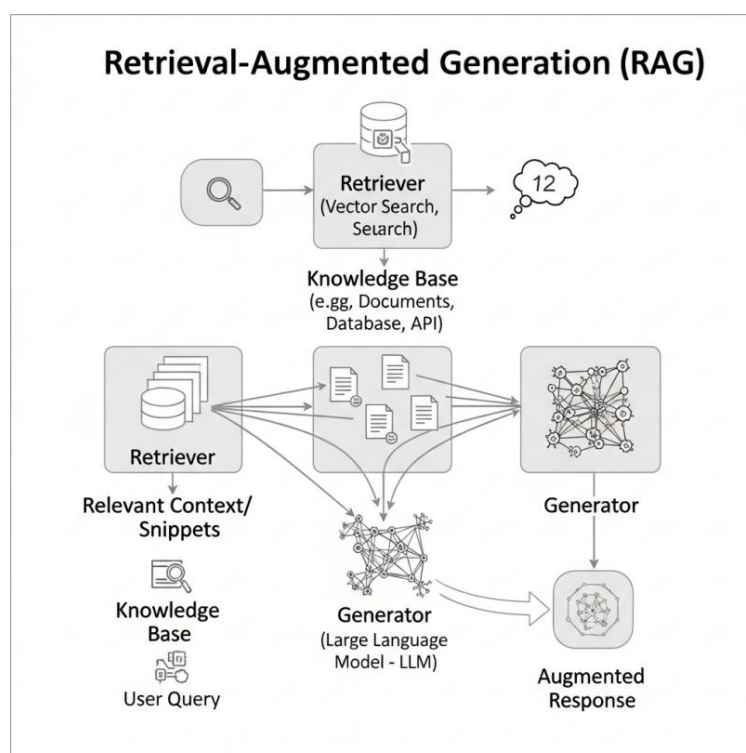
Một cuộc cách mạng thực sự đã đến cùng với sự phát triển của học sâu (Deep Learning) và kiến trúc Transformer, được giới thiệu lần đầu trong công trình “Attention is All You Need” (Vaswani et al., 2017). Sự ra đời của các mô hình ngôn ngữ được huấn luyện trước (pre-trained language models - PLMs) như BERT (Devlin et al., 2018) với cơ chế mã hóa hai chiều đã giúp máy tính lần đầu tiên có thể “hiểu” được một từ trong mối quan hệ ngữ cảnh với toàn bộ câu văn xung quanh nó. Các mô hình thế hệ sau như GPT và T5 tiếp tục đẩy xa giới hạn của việc sinh ngôn ngữ tự nhiên, mở ra tiềm năng khổng lồ cho các hệ thống hỏi-đáp tự động.



Hình 3. Sơ đồ kiến trúc Transformer (Nguồn: Vaswani et al., 2017)

2.4. Thế hệ mới - Truy xuất tăng cường cho sinh ngôn ngữ (RAG)

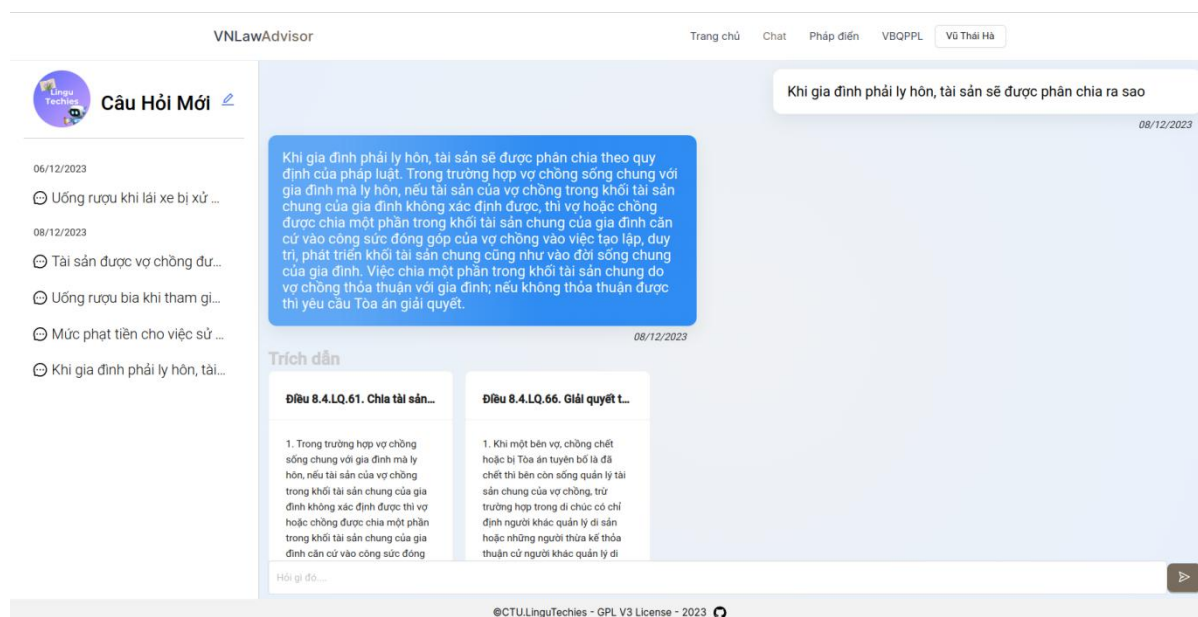
Mặc dù các mô hình ngôn ngữ lớn rất mạnh mẽ, chúng vẫn gặp phải vấn đề ảo giác (hallucination) - tức là tạo ra thông tin không chính xác. Để giải quyết vấn đề này, đặc biệt trong các lĩnh vực yêu cầu tính chính xác tuyệt đối như pháp luật, kỹ thuật Retrieval-Augmented Generation (RAG), được giới thiệu trong bài báo của Lewis và cộng sự (2020), đã trở thành một hướng tiếp cận đột phá. Nguyên tắc của RAG là kết hợp một module truy xuất (Retriever) để tìm các đoạn văn bản liên quan và một module sinh (Generator) để tạo ra câu trả lời dựa trên thông tin đã truy xuất được. Cách tiếp cận này giúp “neo” câu trả lời vào các bằng chứng xác thực, giảm thiểu thông tin sai lệch và cho phép hệ thống trích dẫn nguồn gốc rõ ràng.



Hình 4. Sơ đồ kỹ thuật RAG

Trên thế giới, nhiều dự án và hệ thống AI hỗ trợ pháp luật đã được phát triển nhằm tự động hóa quá trình tra cứu và phân tích văn bản pháp luật. Ví dụ, Siren Analytics sử dụng kỹ thuật RAG kết hợp mô hình ngôn ngữ lớn để tự động trích xuất và phân tích các văn bản luật quốc tế, hỗ trợ luật sư và chuyên gia pháp lý trong việc nghiên cứu và tư vấn. Các AI Expert Agents chuyên sâu về luật quốc tế cũng được triển khai để phân tích công ước, án lệ và tài liệu pháp lý phức tạp, giúp nâng cao hiệu quả nghiên cứu và ra quyết định. Ngoài ra, các hội thảo quốc tế như Legal Information Retrieval meets Artificial Intelligence (LIRAI) tập trung phát triển các phương pháp kết hợp AI và truy xuất thông tin pháp lý, giải quyết các thách thức về độ chính xác và khả năng giải thích trong môi trường phức tạp. Một số chatbot pháp lý tiên tiến cũng áp dụng kỹ thuật RAG để cung cấp câu trả lời chính xác dựa trên tài liệu pháp luật chính thống, đồng thời hạn chế lỗi do mô hình sinh thông tin không chính xác (hallucination).

Tại Việt Nam, các mô hình ngôn ngữ thuần Việt như PhoBERT, ViT5 và Vietnamese-SBERT đã được phát triển, hỗ trợ hiệu quả trong các tác vụ xử lý văn bản tiếng Việt. Một ví dụ điển hình là dự án VN-Law-Advisor do nhóm CTU-LinguTechies phát triển, sử dụng mô hình phoGPT kết hợp kỹ thuật RAG và cơ sở dữ liệu vector Chroma để cung cấp hệ thống hỏi đáp pháp luật tự động, hỗ trợ đại biểu và cán bộ tra cứu, đối chiếu thông tin pháp luật một cách nhanh chóng và chính xác, đồng thời trích dẫn rõ ràng các điều khoản liên quan.



Hình 5. Giao diện ứng dụng tra cứu, hỏi đáp pháp luật của dự án VN-Law-Advisor
(Nguồn: <https://github.com/CTU-LinguTechies/VN-Law-Advisor>)

Dự án VN-Law-Advisor đã xây dựng thành công một hệ thống tra cứu pháp luật toàn diện dựa trên Bộ pháp điển và các văn bản quy phạm pháp luật từ năm 1990 đến nay, được cập nhật liên tục. Hệ thống sử dụng kiến trúc microservices hiện đại, tích hợp các công cụ xử lý PDF (PyMuPDF), tiền xử lý ngôn ngữ tiếng Việt (VnCoreNLP, Underthesea) và lưu trữ embedding trong cơ sở dữ liệu vector Chroma để truy xuất nhanh và chính xác. Người dùng có thể đặt câu hỏi tự nhiên, hệ thống sẽ tự động truy xuất thông tin liên quan và sinh câu trả lời mạch lạc, kèm trích dẫn điều khoản cụ thể, giúp đại biểu và cán bộ hỗ trợ trong các buổi tiếp xúc cử tri có thể phản hồi nhanh chóng, chính xác và có căn cứ pháp lý vững chắc.

3. Mục tiêu đề tài

Mục tiêu của đề tài là xây dựng một hệ thống hỏi - đáp tự động có khả năng tiếp nhận đầu vào là các văn bản pháp luật tiếng Việt dưới định dạng tập tin PDF, phân tích nội dung bằng các kỹ thuật xử lý ngôn ngữ tự nhiên, và sinh ra câu trả lời phù hợp với câu hỏi người dùng. Hệ thống hướng đến việc xử lý tốt các đặc thù của văn bản pháp lý như ngôn ngữ chuyên ngành, cấu trúc phi chuẩn, độ dài lớn và yêu cầu chính xác cao. Đồng thời, đề tài cũng tập trung vào việc tích hợp các mô hình ngôn ngữ tiên tiến nhằm nâng cao khả năng truy xuất thông tin theo ngữ cảnh, phục vụ hiệu quả cho các tình huống thực tiễn như tra cứu quy định pháp luật, tham vấn chính sách hoặc hỗ trợ công tác tiếp xúc cử tri. Mục tiêu cuối

cùng là góp phần tạo ra một giải pháp có tính ứng dụng cao, giúp rút ngắn thời gian tìm kiếm và cải thiện khả năng tiếp cận thông tin pháp luật cho người dân và cán bộ chuyên môn.

4. Đối tượng và phạm vi nghiên cứu

4.1. Đối tượng nghiên cứu:

Đối tượng nghiên cứu của đề tài là các phương pháp và mô hình trong lĩnh vực xử lý ngôn ngữ tự nhiên (NLP), đặc biệt là tập trung vào bài toán hỏi - đáp tự động (Question Answering) trên ngữ liệu tiếng Việt. Ngoài ra, đề tài còn nghiên cứu kỹ thuật xử lý tài liệu đầu là các tập tin PDF văn bản pháp luật, bao gồm trích xuất thông tin và lưu trữ có cấu trúc để phục vụ cho việc truy xuất và sinh câu trả lời.

4.2. Phạm vi nghiên cứu:

Đề tài tập trung vào việc xây dựng một hệ thống có khả năng:

- Nhận diện và trích xuất nội dung từ tài liệu văn bản pháp luật tiếng Việt dưới định dạng tập tin PDF.
- Lưu trữ và tổ chức thông tin pháp lý đã trích xuất để hỗ trợ tìm kiếm và truy vấn hiệu quả.
- Phát triển một chatbot có khả năng tự động sinh câu trả lời cho các câu hỏi pháp luật dựa trên dữ liệu đã lưu trữ.

Hệ thống được triển khai thử nghiệm trên tập hợp các văn bản quy phạm pháp luật phổ biến (như nghị định, nghị quyết, thông tư, luật, ...) với phạm vi ngôn ngữ tiếng Việt. Đề tài không đi sâu vào các vấn đề như diễn giải pháp lý hoặc tư vấn luật chuyên sâu.

5. Phương pháp nghiên cứu

- **Thu thập và tiền xử lý dữ liệu:** Thu thập các văn bản quy phạm pháp luật (loại "Chỉ thị") dạng PDF từ Cổng Thông tin điện tử Chính phủ. Xây dựng một quy trình tiền xử lý chuyên sâu bao gồm: trích xuất văn bản từ file PDF, làm sạch dữ liệu nhiễu, và thực hiện phân đoạn theo ngữ nghĩa (semantic chunking) để tái tạo cấu trúc logic của văn bản.
- **Xây dựng và cài đặt giải pháp:** Thiết kế và cài đặt hệ thống hỏi-đáp dựa trên kiến trúc RAG. Quá trình này bao gồm việc nhúng các đoạn văn bản đã xử lý thành vector bằng mô hình text-embedding-004, lưu trữ và lập chỉ mục trong cơ sở dữ liệu vector ChromaDB. Xây dựng module truy xuất thông tin và tích hợp Mô hình Ngôn ngữ Lớn Gemini để tổng hợp và sinh câu trả lời.
- **Phát triển ứng dụng:** Nghiên cứu và xây dựng một ứng dụng web hoàn chỉnh sử dụng Flask cho phần backend và HTML, CSS, JavaScript cho giao diện người dùng. Tích hợp hệ thống RAG vào backend để xử lý các truy vấn từ người dùng một cách trực quan.
- **Kiểm thử và đánh giá:** Đánh giá hiệu suất của hệ thống một cách định lượng bằng framework RAGAS thông qua các chỉ số như Faithfulness và Context Precision. Thực hiện đánh giá định tính qua các trường hợp sử dụng thực tế để kiểm tra khả năng xử lý các loại câu hỏi khác nhau của hệ thống.

6. Kết quả đạt được:

Xây dựng thành công hệ thống trợ lý ảo hỏi-đáp văn bản quy phạm pháp luật (Chỉ thị), nổi bật với giao diện thân thiện và hiện đại. Hệ thống đã chứng minh được hiệu suất cao, cung cấp các câu trả lời chính xác và phù hợp với ngữ cảnh từ truy vấn của người dùng.

7. Bố cục quyển báo cáo

Phần 1. Phần giới thiệu

Phần 2. Phần nội dung

Chương 1: Mô tả bài toán.

Chương 2: Thiết kế và cài đặt giải pháp

Chương 3: Kiểm thử và đánh giá

Phần 3. Phần kết luận và hướng phát triển.

II. PHẦN NỘI DUNG

CHƯƠNG 1. MÔ TẢ BÀI TOÁN

1.1. Mô tả chi tiết bài toán

Bài toán nghiên cứu được xác định từ nhu cầu thực tiễn trong hoạt động lập pháp và giám sát, đặc biệt là khi phải xử lý các văn bản pháp luật phức tạp, có ảnh hưởng sâu rộng đến người dân. Trong các buổi làm việc hoặc tiếp xúc cử tri, các đại biểu và cán bộ hỗ trợ thường xuyên phải đối mặt với những câu hỏi chính sách đòi hỏi câu trả lời không chỉ nhanh chóng mà còn phải có căn cứ pháp lý rõ ràng. Tuy nhiên, việc tra cứu và đối chiếu thông tin tức thời từ nhiều nguồn văn bản dài và có ngôn ngữ chuyên ngành là một thách thức lớn, ảnh hưởng đến hiệu quả công việc. Bài toán đặt ra là phải xây dựng một hệ thống trợ lý ảo pháp lý, hoạt động như một "chuyên gia" có khả năng đối thoại, phân tích và tổng hợp thông tin một cách thông minh để giải quyết vấn đề này.

Bài toán đặt ra là phải xây dựng một hệ thống trợ lý ảo pháp lý hoạt động như một "chuyên gia", được trang bị một kho kiến thức chuyên sâu và toàn diện về vấn đề sáp nhập. Trải nghiệm của người dùng được thiết kế như một cuộc đối thoại trực tiếp, nơi họ có thể đặt câu hỏi và nhận về câu trả lời đánh tin cậy mà không cần thực hiện bất kỳ thao tác kỹ thuật phức tạp nào.

Hệ thống được trang bị sẵn một kho kiến thức thống nhất, đã được nạp và xử lý từ trước. Kho kiến thức này bao gồm các văn bản chung của nhà nước như Luật, Chỉ thị, và Công điện do các cơ quan trung ương ban hành. Điều này đảm bảo trợ lý ảo có khả năng cung cấp các câu trả lời dựa trên nền tảng pháp lý chung và chính thống của quốc gia. Người dùng cuối có thể tải lên các tập tin văn bản quy phạm pháp luật dạng PDF để truy vấn thông tin trong văn bản đó.

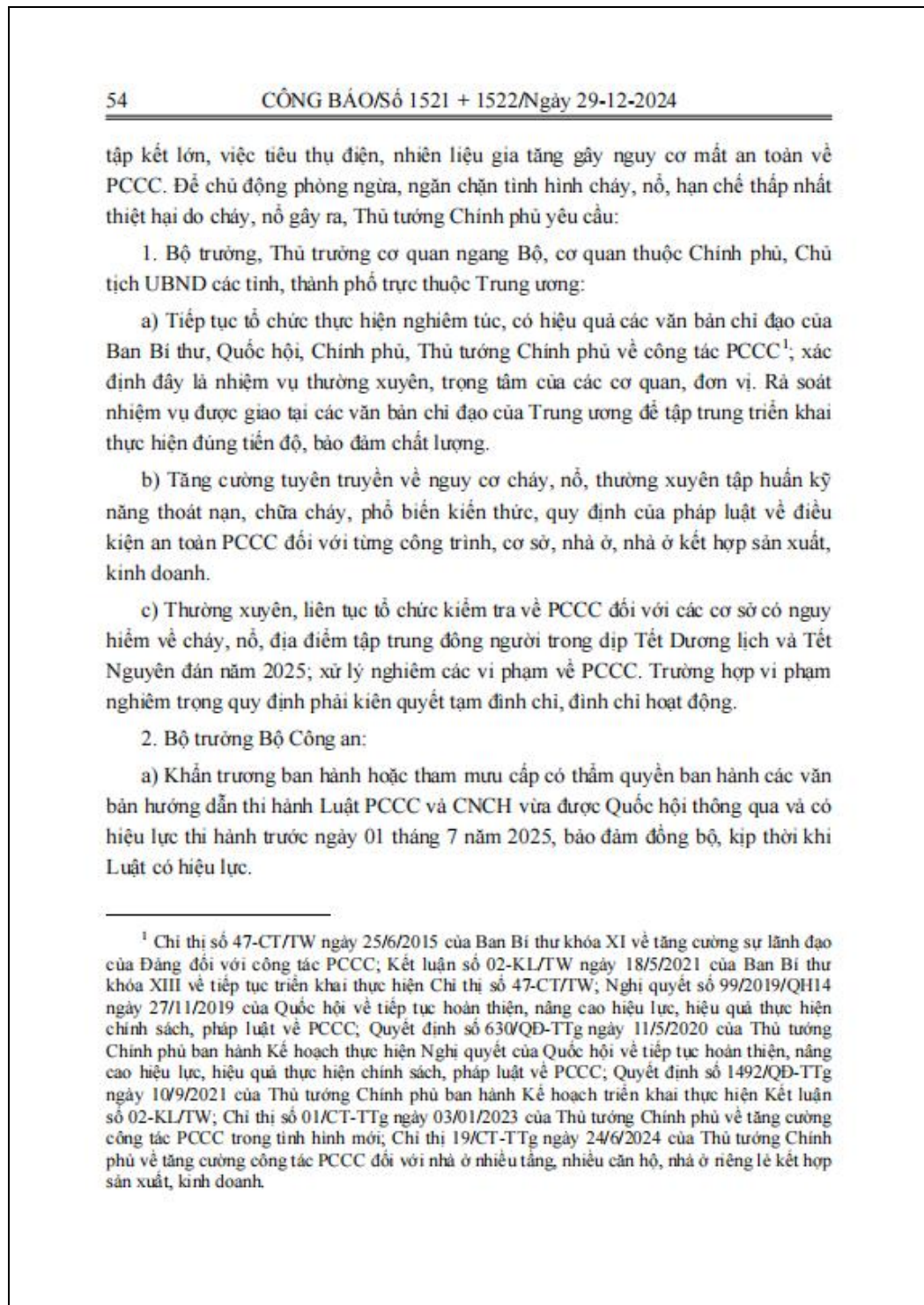
Khi người dùng nhập câu hỏi, hệ thống sẽ tự động phân tích ngữ nghĩa của câu hỏi đó. Thay vì tìm kiếm từ khóa một cách máy móc, nó sẽ thực hiện một cuộc tìm kiếm thông minh trong toàn bộ kho trí thức đã được số hóa để tìm ra những điều khoản, quy định chính xác nhất liên quan đến vấn đề người dùng quan tâm. Các ngữ cảnh phù hợp nhất sau đó sẽ được một mô hình ngôn ngữ lớn tổng hợp và diễn đạt lại thành một câu trả lời tự nhiên hơn, dễ hiểu thông qua kỹ thuật RAG.

Câu trả lời cuối cùng được trình bày như một bản tư vấn hoàn chỉnh, luôn bao gồm hai phần:

- Một phần giải thích chi tiết, cung cấp đầy đủ bối cảnh và lập luận.
- Một phần là nguồn trích dẫn, chỉ rõ thông tin được lấy từ điều khoản nào, của văn bản nào?

Phần trích dẫn này cho phép người dùng xem và kiểm chứng ngay lập tức với văn bản gốc, tạo ra sự minh bạch và tin cậy tuyệt đối. Trong trường hợp không tìm thấy thông tin phù hợp, hệ thống được thiết kế để phản hồi một cách trung thực, đảm bảo không "sáng tạo" thông tin sai lệch.

Ví dụ, với văn bản là **Chỉ thị số 46/CT-TTg** được thể hiện trong hình bên dưới, hệ thống có thể xử lý một truy vấn của người dùng như sau:



*Hình 6. Ví dụ về một văn bản quy phạm pháp luật đầu vào cho hệ thống
(Nguồn: Trang thứ 2 của chỉ thị 46/CT-TTg năm 2024 về việc ngăn ngừa cháy, nổ tại các địa điểm tập trung đông người dịp Tết Dương lịch và Tết Nguyên Đán năm 2025)*

Câu hỏi của người dùng: Theo chỉ thị 46/CT-TTg, Bộ trưởng có những trách nhiệm chính nào trong việc chủ động phòng ngừa, ngăn chặn tình trạng cháy, nổ?

Phản trả lời của trợ lý ảo trong thực tế sẽ cung cấp đầy đủ thông tin từ văn bản gốc, bên dưới chỉ minh họa lại cách mà trợ lý ảo trả lời. Câu trả lời của trợ lý ảo có thể như sau:

“Dựa trên thông tin được cung cấp, Chỉ thị 46/CT-TTg quy định các Bộ trưởng có những trách nhiệm chung và trách nhiệm cụ thể theo từng ngành để chủ động phòng ngừa, ngăn chặn tình trạng cháy, nổ.

Các trách nhiệm chính của các Bộ trưởng:

- ***Thực hiện nghiêm túc các văn bản chỉ đạo về PCCC:*** Tiếp tục tổ chức thực hiệnbảo đảm chất lượng. (Nguồn: Mục 1, khoản a).
- ***Tăng cường tuyên truyền và tập huấn:*** Tăng cường tuyên truyền nhà ở kết hợp sản xuất, kinh doanh. (Nguồn: Mục 1, khoản b).
- ***Tổ chức kiểm tra và xử lý vi phạm:*** Thường xuyên, liên tục tổ chức kiểm tra đình chỉ hoạt động (Nguồn: Mục 1, khoản c).

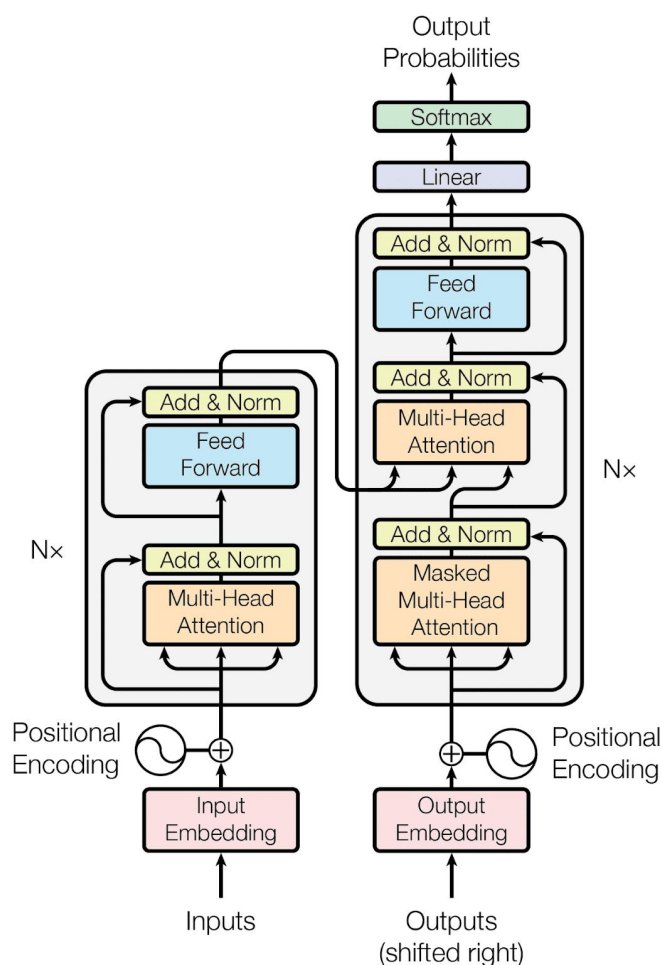
Nguồn trích dẫn:

- Văn bản: Chỉ thị 46/CT-TTg.
- Ngày ban hành: 23/12/2024.
- Người ký: Phạm Minh Chính, THỦ TƯỚNG.”

1.2. Vấn đề và giải pháp liên quan đến bài toán

1.2.1. Kiến trúc Transformer:

Kiến trúc Transformer được giới thiệu lần đầu tiên trong bài báo nổi tiếng “Attention is All You Need” của Vaswani và cộng sự năm 2017. Nó đã tạo nên một cuộc cách mạng trong lĩnh vực xử lý ngôn ngữ tự nhiên (NLP) và nhanh chóng trở thành nền tảng cho hầu hết các mô hình ngôn ngữ lớn (LLMs) hiện đại như BERT, GPT, T5 hay ViT5. Điểm đặc biệt là nó hoàn toàn dựa trên một cơ chế gọi là “Attention” (Chú ý), loại bỏ hoàn toàn sự cần thiết của các thành phần tuần tự (recurrent layers) như trong RNN hay LSTM, vốn là kiến trúc chủ đạo trước đó cho các bài toán xử lý chuỗi. Điều này cho phép Transformer xử lý song song các phần của chuỗi đầu vào, giúp tăng tốc độ huấn luyện và giải quyết hiệu quả vấn đề phụ thuộc xa (long-range dependencies).



Hình 7. Kiến trúc Transformer (Nguồn: Vaswani et al., 2017)

Transformer ban đầu được thiết kế cho bài toán dịch máy (sequence-to-sequence task) và bao gồm hai thành phần chính: Khối mã hóa (Encoder) và khối giải mã (Decoder). Cả Encoder và Decoder đều được cấu tạo từ nhiều tầng (layers) giống hệt nhau được xếp chồng lên nhau. Trong bài báo gốc, số lượng lớp cho cả Encoder và Decoder là sáu ($N=6$). Khối mã hóa tiếp nhận một chuỗi đầu vào và tạo ra một chuỗi các biểu diễn liên tục, giàu ngữ cảnh (contextualized representations) cho chuỗi đó. Những biểu diễn này nắm bắt ý nghĩa của từng từ trong mối quan hệ với các từ khác trong câu. Khối giải mã tiếp nhận đầu ra của khối mã hóa (các biểu diễn ngữ cảnh) cùng với chuỗi đầu ra đã được tạo ra ở bước trước đó để tạo ra từ tiếp theo trong chuỗi đầu ra một cách tự hồi quy.

1.2.2. Mô hình ngôn ngữ lớn (Large Language Models - LLMs):

Mô hình ngôn ngữ lớn (LLMs) là một loại mô hình trí tuệ nhân tạo được huấn luyện trên một khối lượng dữ liệu văn bản khổng lồ, cho phép chúng có khả năng hiểu, tóm tắt, tạo sinh và dự đoán nội dung ngôn ngữ của con người một cách tinh vi. Nền tảng kiến trúc của hầu hết các LLM hiện đại đều dựa trên kiến trúc Transformer, giúp chúng xử lý hiệu quả các mối quan hệ ngữ nghĩa phức tạp và các phụ thuộc xa trong văn bản.

Về cơ bản, LLM hoạt động bằng cách học các mẫu (patterns), cấu trúc ngữ pháp và các mối liên hệ về ngữ nghĩa từ hàng tỷ câu văn. Dựa trên những gì đã học được, nhiệm vụ

cốt lõi của chúng là dự đoán từ tiếp theo có khả năng xuất hiện cao nhất trong một chuỗi văn bản. Bằng cách lặp lại quá trình này, chúng có thể tạo ra các đoạn văn, bài luận, mã nguồn, hoặc các câu trả lời mạch lạc và tự nhiên.

Dựa trên việc sử dụng các thành phần của kiến trúc Transformer, các LLM thường được chia thành ba loại chính:

- **Mô hình chỉ bao gồm khối mã hóa (Encoder-only):** Tiêu biểu là BERT và các biến thể của nó. Các mô hình này được thiết kế để xây dựng một biểu diễn ngữ nghĩa sâu sắc cho toàn bộ câu văn. Chúng có khả năng "nhìn" được cả ngữ cảnh bên trái và bên phải của một từ, do đó cực kỳ mạnh mẽ trong các tác vụ yêu cầu sự "hiểu" sâu như phân loại văn bản, nhận dạng thực thể, hoặc phân tích cảm xúc.
- **Mô hình chỉ bao gồm khối giải mã (Decoder-only):** Tiêu biểu là dòng mô hình GPT. Các mô hình này hoạt động theo cơ chế tự hồi quy, tức là chúng dự đoán từ tiếp theo dựa trên các từ đã được sinh ra trước đó. Điều này làm cho chúng trở nên vượt trội trong các tác vụ tạo sinh văn bản một cách tự do, chẳng hạn như viết truyện, soạn email, hoặc đóng vai trò là một chatbot đối thoại.
- **Mô hình bao gồm cả khối mã hóa và giải mã (Encoder-Decoder):** Tiêu biểu là T5 hay ViT5. Các mô hình này được thiết kế cho các bài toán "đầu vào-đầu ra" (sequence-to-sequence), nơi chúng nhận một chuỗi đầu vào, mã hóa nó thành một biểu diễn ngữ nghĩa, sau đó giải mã biểu diễn đó thành một chuỗi đầu ra mới. Chúng rất hiệu quả trong các tác vụ như dịch máy, tóm tắt văn bản, và trả lời câu hỏi.

Mô hình sinh ngôn ngữ được sử dụng trong đề tài này là **Gemini**. Giống như dòng mô hình GPT, các mô hình trong họ Gemini chủ yếu được xây dựng dựa trên kiến trúc chỉ bao gồm khối giải mã (Decoder-only). Cách tiếp cận này giúp tối ưu hóa khả năng tạo sinh văn bản một cách mạch lạc và tự nhiên, rất phù hợp cho các tác vụ đối thoại và tổng hợp thông tin như trong hệ thống hỏi-đáp.

Phát huy thế mạnh của kiến trúc này, luận văn lựa chọn mô hình **Gemini 2.5 Pro** để đảm nhận vai trò “Generator” trong hệ thống RAG. Đây là một phiên bản tiên tiến do Google phát triển, được chọn lựa dựa trên các ưu điểm nổi bật sau:

- **Hiệu suất vượt trội:** **Gemini Pro** được thiết kế cho các tác vụ phức tạp, có khả năng suy luận đa bước và hiểu sâu sắc các ngữ cảnh phức tạp. Điều này đặc biệt quan trọng khi tổng hợp thông tin từ nhiều điều khoản pháp luật khác nhau để đưa ra một câu trả lời chính xác.
- **Cửa sổ ngữ cảnh cực lớn (Large Context Window):** **Gemini Pro** hỗ trợ một cửa sổ ngữ cảnh lên đến hàng triệu token, cho phép nó xử lý và phân tích đồng thời một lượng lớn văn bản pháp luật được truy xuất. Khả năng này đảm bảo mô hình không bỏ sót thông tin quan trọng và có thể đối chiếu các quy định từ nhiều nguồn một cách hiệu quả.
- **Sự cân bằng giữa chất lượng và hiệu quả:** Mặc dù mạnh mẽ, **Gemini Pro** vẫn cung cấp một sự cân bằng tối ưu giữa năng lực xử lý cao cấp và chi phí vận hành. Đối với một mục tiêu yêu cầu độ chính xác và suy luận pháp lý cao, đây

là lựa chọn phù hợp để đảm bảo chất lượng đầu ra mà vẫn nằm trong giới hạn tài nguyên cho phép.

Trong hệ thống RAG của tôi, Gemini sẽ không trả lời dựa trên kiến thức nội tại của nó. Thay vào đó, nó sẽ nhận câu hỏi từ người dùng cùng với các đoạn văn bản pháp luật liên quan được truy xuất từ cơ sở dữ liệu vector. Nhiệm vụ của nó là đóng vai trò như một “bộ não suy luận”, đọc hiểu các ngữ cảnh được cung cấp và tổng hợp chúng thành một câu trả lời cuối cùng, mạch lạc, chính xác và bám sát vào nguồn thông tin pháp lý đã cho.

1.2.3. Hệ thống hỏi đáp (Question Answering - QA):

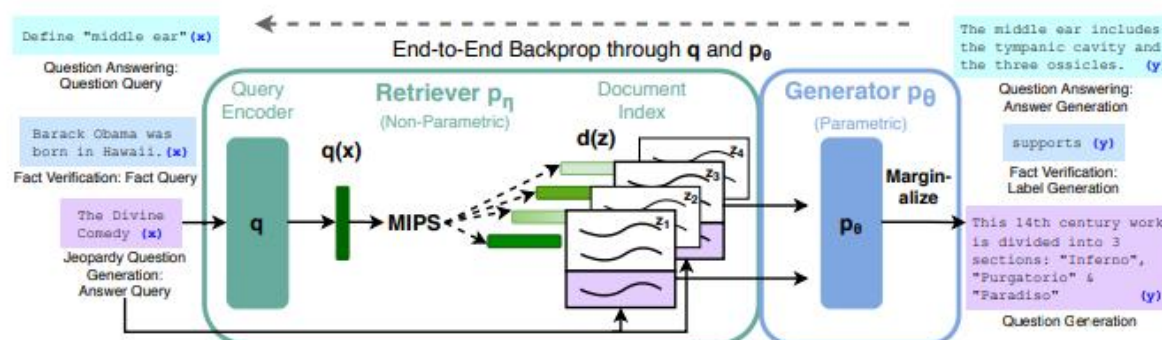
Hệ thống Hỏi-đáp (Question Answering - QA) là một lĩnh vực chuyên sâu trong Xử lý Ngôn ngữ Tự nhiên (NLP) với mục tiêu chính là xây dựng các hệ thống có khả năng tự động trả lời các câu hỏi do con người đặt ra bằng ngôn ngữ tự nhiên. Khác với các công cụ tìm kiếm truyền thống chỉ trả về một danh sách các tài liệu có thể liên quan, một hệ thống QA lý tưởng sẽ cung cấp một câu trả lời ngắn gọn, chính xác và trực tiếp cho câu hỏi, giúp người dùng tiết kiệm thời gian và công sức phân tích.

Các hệ thống QA ban đầu thường dựa trên các phương pháp trích xuất thông tin (Information Extraction) và các mẫu được định sẵn. Cùng với sự phát triển của học máy, các mô hình phức tạp hơn đã ra đời, có khả năng đọc và hiểu một đoạn văn bản để tìm ra câu trả lời. Các kiến trúc tiên phong trong giai đoạn này, chẳng hạn như DrQA (Document Reader for Question Answering) hay BiDAF (Bi-Directional Attention Flow), đã đạt được những thành công đáng kể trong việc xác định chính xác "span" (một đoạn văn bản) chứa câu trả lời trong một tài liệu cho trước. Dạng QA này được gọi là "Hỏi-đáp trích xuất" (Extractive QA).

Tuy nhiên, hạn chế của các hệ thống trích xuất là chúng chỉ có thể lấy ra thông tin đã tồn tại y nguyên trong văn bản mà không có khả năng tổng hợp, diễn giải hay trả lời các câu hỏi phức tạp đòi hỏi suy luận từ nhiều nguồn. Một bước đột phá thực sự đã đến cùng với sự trỗi dậy của các Mô hình Ngôn ngữ Lớn (LLMs). Các hệ thống QA hiện đại không còn chỉ trích xuất, mà đã có thể tạo sinh ra các câu trả lời hoàn toàn mới, mạch lạc và tự nhiên hơn.

Kỹ thuật Retrieval-Augmented Generation (RAG) chính là một kiến trúc tiêu biểu cho thế hệ hệ thống QA này. Bằng cách kết hợp một module truy xuất thông tin (Retriever) và một module tạo sinh (Generator) dựa trên LLM, hệ thống RAG có thể cung cấp những câu trả lời vừa bám sát vào nguồn kiến thức xác thực, vừa được diễn đạt một cách tự nhiên, phù hợp với ngữ cảnh của câu hỏi.

1.2.4. Retrieval-augmented Generation (RAG):



Hình 8. Kiến trúc kỹ thuật RAG (Nguồn: Lewis et al., 2020)

Retrieval-Augmented Generation (RAG) là một khung trí tuệ nhân tạo hoặc một kỹ thuật học máy được giới thiệu lần đầu tiên trong bài báo “*Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*” (Lewis et al., 2020) nhằm nâng cao khả năng của các mô hình ngôn ngữ lớn (LLM) bằng cách tích hợp chúng với khả năng truy xuất thông tin từ bên ngoài. Điều này có nghĩa là LLM không còn chỉ dựa vào dữ liệu huấn luyện tĩnh của chúng nữa. Nguyên tắc cốt lõi của RAG là “neo” các LLM vào thông tin chính xác, cập nhật nhất từ một cơ sở tri thức bên ngoài trước khi tạo ra phản hồi. Cách tiếp cận này giúp giải quyết “khoảng trống kiến thức” trong cách hoạt động của các LLM truyền thống.

Động lực chính đằng sau sự phát triển của RAG bao gồm việc cho phép tạo ra văn bản chính xác, cập nhật và có thể kiểm chứng được. Quan trọng hơn, RAG giải quyết một số hạn chế chính của LLM, chẳng hạn như việc tạo ra “ảo giác” (hallucination) - tức là thông tin nghe có vẻ hợp lý nhưng thực tế lại không chính xác hoặc gây hiểu lầm. Bằng cách neo quá trình tạo sinh vào các tài liệu được truy xuất và đã được xác minh, RAG giảm thiểu việc tạo ra nội dung sai lệch. Ngoài ra, RAG còn cung cấp cho người dùng cái nhìn sâu sắc về quy trình tạo sinh của LLM bằng cách liên kết các phản hồi với thông tin nguồn.

Hơn nữa, động lực đằng sau RAG không chỉ là dừng lại ở việc đạt được độ chính xác thực tế; nó còn hướng đến việc xây dựng niềm tin và khả năng kiểm chứng trong nội dung do AI tạo ra, điều này rất quan trọng đối với việc áp dụng trong doanh nghiệp và các ứng dụng quan trọng. Hiện tượng ảo giác của LLM là một rào cản lớn đối với niềm tin. RAG nhằm mục đích giảm thiểu những điều này bằng cách đưa các phản hồi vào các tài liệu có thể kiểm chứng. Khả năng truy xuất nguồn gốc của nội dung được tạo ra từ một nguồn sự thật là một kết quả then chốt. Khả năng truy vết này giúp củng cố niềm tin của người dùng vào giải pháp AI.

Từ góc độ vĩ mô, các hệ thống RAG có thể được chia thành hai thành phần chính:

- Phần truy xuất (Retriever): Bao gồm các hoạt động đa dạng như tiền xử lý dữ liệu bên ngoài, các cơ chế truy xuất dày đặc (dense) hoặc thưa thớt (sparse) để tìm kiếm tài liệu liên quan, và có thể bao gồm cả việc xếp hạng lại (reranking) và tỉa bớt (pruning) các tài liệu này. Thành phần truy xuất sẽ tìm kiếm trong các cơ sở dữ liệu / nguồn bên ngoài dựa trên truy vấn người dùng.

- **Phân tạo sinh:** Bao gồm các thành phần như lập kế hoạch truy xuất (quyết định cái gì / làm thế nào để truy xuất), tích hợp kiến thức từ nhiều nguồn, và suy luận logic bởi LLM để tổng hợp một phản hồi dựa trên câu lệnh đã được tăng cường. Mô-đun tạo sinh xử lý thông tin được truy xuất để tạo ra văn bản giống như con người.

Ngoài ra, các yếu tố liên kết khác bao gồm việc phân đoạn tài liệu (document chunking), tạo vector nhúng (embedding generation), và các cơ chế đảm bảo an ninh và độ tin cậy.

Bảng 1. Các thành phần cốt lõi của hệ thống RAG

Thành phần	Mô tả / Chức năng
Nguồn dữ liệu	Kho lưu trữ kiến thức bên ngoài mà RAG sẽ truy xuất thông tin.
Bộ chỉ mục / Bộ nhúng (Indexer / Embedder)	Xử lý và chuyển đổi dữ liệu nguồn thành các biểu diễn số (vector nhúng) và lưu trữ chúng trong một cơ sở dữ liệu vector để tìm kiếm hiệu quả.
Bộ truy xuất (Retriever)	Tìm kiếm và lấy các đoạn thông tin liên quan nhất từ cơ sở dữ liệu vector dựa trên truy vấn của người dùng.
Bộ tăng cường (Augmenter)	Kết hợp thông tin được truy xuất với truy vấn ban đầu của người dùng để tạo ra một câu lệnh tăng cường cho LLM.
Bộ tạo sinh / LLM (Generator / LLM)	Nhận câu lệnh tăng cường và tạo ra một phản hồi mạch lạc, dựa trên ngữ cảnh, kết hợp cả kiến thức được truy xuất và kiến thức nội tại của nó.

RAG mang lại nhiều ưu điểm đáng kể, nổi bật nhất là khả năng tăng cường độ chính xác và giảm thiểu tình trạng “ảo giác” của mô hình ngôn ngữ lớn bằng cách truy xuất thông tin thực tế từ các nguồn dữ liệu bên ngoài. Điều này cũng cho phép RAG linh hoạt cập nhật kiến thức mới mà không cần huấn luyện lại toàn bộ mô hình, đồng thời tăng tính minh bạch khi câu trả lời thường đi kèm nguồn trích dẫn để người dùng kiểm chứng, và cải thiện khả năng xử lý kiến thức chuyên ngành hiệu quả. Tuy nhiên, RAG cũng tồn tại một số hạn chế. Hiệu quả của nó phụ thuộc lớn vào chất lượng của nguồn dữ liệu được truy xuất; nếu dữ liệu không tốt, kết quả cũng sẽ bị ảnh hưởng. Bên cạnh đó, việc triển khai RAG làm tăng độ phức tạp của hệ thống và có thể gây ra độ trễ trong thời gian phản hồi do quá trình truy xuất. Cuối cùng, luôn có nguy cơ cơ chế truy xuất không chọn được thông tin tối ưu hoặc bị ảnh hưởng bởi các thiên kiến tiềm ẩn trong dữ liệu. Mặc dù vậy, RAG vẫn là một giải pháp mạnh mẽ để nâng cao chất lượng và độ tin cậy của LLM, nhưng cần cân nhắc kỹ về nguồn dữ liệu và độ phức tạp khi triển khai.

1.2.5. Nhúng vector (Vector embedding):

Vector embedding (Nhúng vector) là một kỹ thuật nền tảng trong Xử lý Ngôn ngữ Tự nhiên (NLP) hiện đại, với nhiệm vụ chuyển đổi các đơn vị văn bản (như từ, câu, hoặc đoạn văn) thành các biểu diễn dạng số, cụ thể là các vector trong một không gian đa chiều. Mục tiêu chính của kỹ thuật này là mã hóa **ngữ nghĩa** của văn bản vào một dạng mà máy tính có thể hiểu và xử lý được về mặt toán học.

Trong không gian vector này, các từ hoặc các câu có ý nghĩa tương tự nhau sẽ được biểu diễn bởi các vector nằm gần nhau. Ví dụ, vector của từ "luật sư" sẽ nằm gần vector của từ "thẩm phán" hơn là so với vector của từ "bác sĩ". Mối quan hệ ngữ nghĩa này không chỉ giới hạn ở sự tương đồng, mà còn có thể thể hiện các mối quan hệ phức tạp khác như quan hệ nhân quả hay đối lập. Quá trình tạo ra các vector này thường được thực hiện bởi các mô hình ngôn ngữ lớn đã được huấn luyện trước trên một kho dữ liệu khổng lồ, cho phép chúng học được các sắc thái tinh tế của ngôn ngữ.

Trong kiến trúc RAG của luận văn này, vector embedding đóng hai vai trò then chốt. **Thứ nhất**, trong giai đoạn xử lý dữ liệu, toàn bộ các đoạn văn bản từ kho kiến thức pháp luật sẽ được một mô hình embedding chuyển đổi thành các vector và lưu trữ trong cơ sở dữ liệu vector ChromaDB. **Thứ hai**, trong giai đoạn vận hành, khi người dùng đặt một câu hỏi, câu hỏi đó cũng sẽ được chuyển đổi thành một vector bằng chính mô hình embedding đó. Vector của câu hỏi sau đó sẽ được sử dụng để tìm kiếm các vector văn bản gần nhất (tương đồng nhất về mặt ngữ nghĩa) trong ChromaDB, từ đó truy xuất ra những ngữ cảnh phù hợp nhất để trả lời.

Hệ thống sử dụng mô hình text-embedding-004 của Google, một mô hình mạnh mẽ và hiệu quả được thiết kế để tạo ra các vector embedding chất lượng cao cho các tác vụ tìm kiếm và truy xuất thông tin.

1.2.6. Cơ sở dữ liệu vector (Vector database/stores):

Cơ sở dữ liệu vector (vector database/stores) là một loại cơ sở dữ liệu được thiết kế chuyên biệt để lưu trữ, quản lý và truy vấn các vector embedding một cách hiệu quả. Không giống như các cơ sở dữ liệu quan hệ truyền thống lưu trữ dữ liệu có cấu trúc trong các bảng, cơ sở dữ liệu vector lưu trữ dữ liệu dưới dạng các vector đa chiều trong một không gian toán học, nơi khoảng cách và hướng của các vector thể hiện mối quan hệ ngữ nghĩa giữa các đối tượng dữ liệu.

Chức năng cốt lõi của một cơ sở dữ liệu vector là khả năng thực hiện các truy vấn tìm kiếm tương đồng (similarity search) với tốc độ cực nhanh. Thay vì tìm kiếm dựa trên sự trùng khớp từ khóa chính xác, nó tìm kiếm các vector "láng giềng" gần nhất với vector của câu hỏi truy vấn. Quá trình này, thường được thực hiện bằng các thuật toán như Approximate Nearest Neighbor (ANN), cho phép hệ thống tìm ra các đoạn văn bản có ý nghĩa tương tự nhau, ngay cả khi chúng không chia sẻ chung một từ khóa nào. Đây chính là nền tảng của tìm kiếm ngữ nghĩa (semantic search), giúp máy tính "hiểu" được ý định của người dùng thay vì chỉ khớp chuỗi ký tự.

Trong hệ thống RAG, cơ sở dữ liệu vector đóng vai trò như một "bộ não" lưu trữ toàn bộ kiến thức đã được số hóa. Mỗi đoạn văn bản từ các tài liệu pháp luật sau khi được chuyển đổi thành vector embedding sẽ được lưu trữ và lập chỉ mục trong cơ sở dữ liệu này. Khi

người dùng đặt câu hỏi, câu hỏi đó cũng được chuyển thành một vector và được dùng để truy vấn, tìm ra các đoạn văn bản có nội dung liên quan nhất để làm ngữ cảnh cho mô hình ngôn ngữ lớn (LLM).

1.2.7. Phương pháp đánh giá hệ thống - RAGAS:

Việc đánh giá chất lượng của một hệ thống Hỏi-đáp dựa trên kiến trúc RAG là một bài toán phức tạp, bởi nó không chỉ phụ thuộc vào chất lượng của câu trả lời cuối cùng mà còn cả chất lượng của bước truy xuất thông tin. Để giải quyết vấn đề này, RAGAS (Retrieval-Augmented Generation Assessment) đã được phát triển như một framework chuyên biệt, cung cấp một bộ các chỉ số đo lường khách quan và toàn diện cho hiệu suất của pipeline RAG.

Điểm đặc biệt của RAGAS là nó hoạt động mà không cần đến một bộ dữ liệu "vàng" (golden dataset) với các câu trả lời mẫu do con người viết tay. Thay vào đó, nó sử dụng chính các mô hình ngôn ngữ lớn (LLMs) để thực hiện việc đánh giá một cách tự động dựa trên câu hỏi, ngữ cảnh được truy xuất và câu trả lời do hệ thống tạo ra. Điều này giúp quá trình đánh giá trở nên linh hoạt và có khả năng mở rộng cao.

Framework RAGAS tập trung vào việc đo lường hai khía cạnh chính của hệ thống: chất lượng của module truy xuất (Retriever) và chất lượng của module sinh (Generator). Các chỉ số đánh giá cốt lõi bao gồm:

- **Faithfulness (Tính trung thực):** Đây là chỉ số quan trọng nhất để chống lại hiện tượng "ảo giác". Nó đo lường mức độ mà câu trả lời do LLM tạo ra hoàn toàn bám sát và có thể suy ra từ các đoạn ngữ cảnh đã được cung cấp. Một điểm số Faithfulness cao cho thấy hệ thống đang trả lời dựa trên bằng chứng xác thực, chứ không bịa đặt thông tin.
- **Answer Relevancy (Độ liên quan của câu trả lời):** Chỉ số này đánh giá xem câu trả lời được tạo ra có thực sự là câu trả lời cho câu hỏi của người dùng hay không. Đôi khi, một câu trả lời có thể trung thực (bám sát ngữ cảnh) nhưng lại không đi thẳng vào vấn đề mà người dùng quan tâm.
- **Context Precision và Context Recall (Độ chính xác và Độ phủ của ngữ cảnh):** Hai chỉ số này dùng để đánh giá trực tiếp chất lượng của module Retriever. **Context Precision** đo lường xem các ngữ cảnh được truy xuất có chứa thông tin thực sự liên quan đến câu hỏi hay không (giảm thiểu "nhiều"). Trong khi đó, **Context Recall** đo lường xem liệu tất cả các thông tin cần thiết để trả lời câu hỏi đã được truy xuất đầy đủ hay chưa.

Việc cung cấp RAGAS sẽ cung cấp những số liệu định lượng, khách quan để đánh giá hiệu quả của hệ thống đã xây dựng. Các điểm số từ RAGAS sẽ là bằng chứng khoa học vững chắc để kết luận về mức độ thành công của mô hình trong việc giải quyết bài toán hỏi-đáp văn bản pháp luật một cách chính xác và đáng tin cậy.

1.2.8. Các thư viện và công cụ hỗ trợ:

1.2.8.1. Requests:

Thư viện **Requests** là một công cụ HTTP client mạnh mẽ và phổ biến bậc nhất trong hệ sinh thái Python, được sử dụng để gửi các loại yêu cầu HTTP một cách đơn giản. Trong

luận văn này, thư viện Requests đóng vai trò then chốt trong quy trình thu thập dữ liệu, cụ thể là để thực hiện việc tải về các tập tin văn bản pháp luật dạng PDF từ Cổng Thông tin điện tử Chính phủ. Sau khi đã xác định được đường dẫn URL chính xác của tập tin, một hàm chuyên dụng sẽ gọi đến Requests để gửi yêu cầu GET đến máy chủ. Đặc biệt, việc sử dụng tham số *stream=True* khi tải file cho phép xử lý các tệp có dung lượng lớn một cách hiệu quả, bằng cách đọc dữ liệu theo từng khối nhỏ thay vì tải toàn bộ về bộ nhớ cùng lúc, giúp tối ưu hóa tài nguyên hệ thống.

1.2.8.2. Selenium:

Selenium là một bộ công cụ mã nguồn mở mạnh mẽ, được thiết kế để tự động hóa các trình duyệt web. Vai trò chính của nó là mô phỏng các hành động của người dùng như nhấp chuột, điền biểu mẫu, và điều hướng qua các trang. Trong khuôn khổ luận văn này, Selenium được sử dụng để điều khiển trình duyệt Chrome, tự động duyệt qua các trang danh sách văn bản trên Cổng Thông tin điện tử Chính phủ, xử lý việc chuyển trang (pagination), và truy cập vào từng trang chi tiết của mỗi văn bản pháp luật. Nó chịu trách nhiệm tải về toàn bộ mã nguồn HTML của mỗi trang, đảm bảo lấy được nội dung đầy đủ trước khi chuyển cho BeautifulSoup để tiến hành bóc tách dữ liệu.

1.2.8.3. BeautifulSoup:

BeautifulSoup là một thư viện Python được thiết kế chuyên biệt cho việc bóc tách dữ liệu từ các tệp HTML và XML. Nhiệm vụ chính của nó là phân tích (parse) một tài liệu HTML thô và tạo ra một cây cấu trúc đối tượng, cho phép các lập trình viên điều hướng và tìm kiếm trong cây phân tích một cách dễ dàng. Trong quy trình thu thập dữ liệu của luận văn, sau khi Selenium đã tải về mã nguồn trang web, BeautifulSoup đảm nhận vai trò then chốt trong việc phân tích cây cấu trúc này. Nó được sử dụng để tìm kiếm và trích xuất các thông tin quan trọng như tiêu đề, số hiệu, ngày ban hành và đường dẫn tải tệp PDF bằng cách định vị các thẻ HTML cụ thể dựa trên thuộc tính của chúng, qua đó đơn giản hóa đáng kể quá trình trích xuất siêu dữ liệu một cách chính xác từ cấu trúc HTML phức tạp.

1.2.8.4. PyMuPDF:

PyMuPDF là một thư viện Python hiệu suất cao, được sử dụng để truy cập và xử lý các tài liệu PDF. Trong luận văn này, nó đóng một vai trò thiết yếu trong giai đoạn tiền xử lý dữ liệu. Sau khi các văn bản pháp luật được thu thập dưới dạng tệp PDF, PyMuPDF được sử dụng để mở và trích xuất toàn bộ nội dung văn bản từ các tệp này một cách chính xác. Nhờ tốc độ xử lý nhanh và độ tin cậy cao, thư viện này đảm bảo rằng dữ liệu văn bản thô được cung cấp cho các bước tiếp theo như phân đoạn và nhúng vector là dữ liệu sạch và toàn vẹn, tạo nền tảng vững chắc cho toàn bộ hệ thống RAG.

1.2.8.5. LangChain:

LangChain là một framework mã nguồn mở mạnh mẽ, được thiết kế để đơn giản hóa việc xây dựng các ứng dụng phức tạp dựa trên Mô hình Ngôn ngữ Lớn (LLMs). Nó hoạt động như một lớp điều phối trung tâm, cung cấp một bộ công cụ và các module được tiêu chuẩn hóa để kết nối các thành phần khác nhau một cách liền mạch. Mục tiêu chính của

LangChain là cho phép các nhà phát triển tạo ra các ứng dụng nhận biết được ngữ cảnh và có khả năng suy luận, thay vì chỉ thực hiện các lệnh gọi API đơn lẻ đến LLM.

1.2.8.6. RAGAS:

RAGAS (Retrieval-Augmented Generation Assessment) là một framework mã nguồn mở được thiết kế chuyên biệt để đánh giá và kiểm thử chất lượng của các hệ thống AI xây dựng theo kiến trúc RAG. Nó được sử dụng để đo lường hiệu suất của toàn bộ pipeline một cách khách quan thông qua các chỉ số quan trọng như **Faithfulness** (đo lường câu trả lời có trung thực với ngữ cảnh không) và **Answer Relevancy** (đo lường câu trả lời có đúng trọng tâm câu hỏi không). Điểm đặc biệt của RAGAS là khả năng hoạt động mà không cần đến bộ dữ liệu câu trả lời mẫu do con người tạo ra; thay vào đó, nó sử dụng chính các mô hình ngôn ngữ lớn để làm "giám khảo", giúp quá trình kiểm thử trở nên linh hoạt, tự động và có khả năng mở rộng cao.

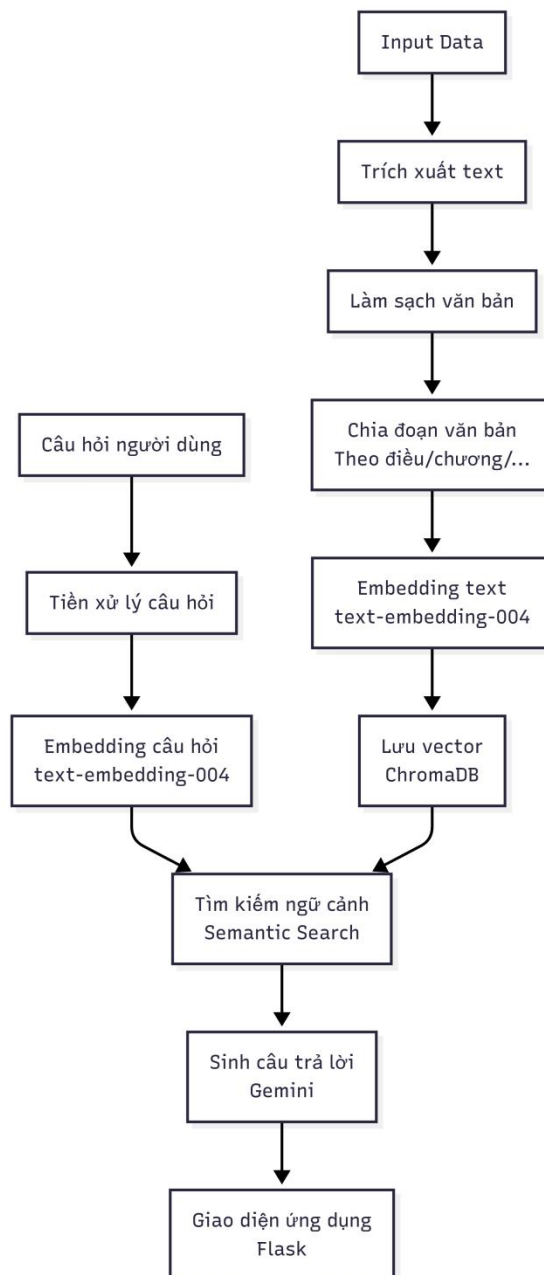
1.2.8.7. Flask:

Flask là một micro web framework mạnh mẽ và linh hoạt được viết bằng Python. Được gọi là "micro" vì nó cung cấp một lõi tối giản nhưng có khả năng mở rộng cao thông qua các extension, cho phép nhà phát triển toàn quyền kiểm soát ứng dụng của mình. Trong luận văn này, Flask đóng vai trò là nền tảng cho toàn bộ hệ thống backend, chịu trách nhiệm xử lý các yêu cầu từ người dùng, quản lý logic nghiệp vụ và cung cấp dữ liệu cho giao diện frontend. Cụ thể, nó được sử dụng để xây dựng các API endpoint, chẳng hạn như /api/chat, để nhận câu hỏi và trả về câu trả lời từ hệ thống RAG. Hơn nữa, Flask được tích hợp với các extension như **Flask-SQLAlchemy** để tương tác với cơ sở dữ liệu (quản lý người dùng, lịch sử hội thoại) và **Flask-Login** để xử lý việc xác thực người dùng, đảm bảo một môi trường làm việc an toàn và cá nhân hóa.

CHƯƠNG 2. THIẾT KẾ VÀ CÀI ĐẶT GIẢI PHÁP

2.1. Thiết kế hệ thống

Kiến trúc tổng thể của hệ thống được thiết kế dựa trên mô hình RAG tiên tiến. Mục tiêu của thiết kế này là xây dựng một trợ lý ảo không chỉ có khả năng sinh ngôn ngữ một cách tự nhiên mà còn phải đảm bảo các câu trả lời luôn được “neo” vào một nguồn kiến thức pháp lý xác thực, giảm thiểu tối đa hiện tượng “ảo giác” (hallucination). Như được minh họa trong hình 13, quy trình hoạt động của hệ thống được chia thành hai luồng chính, song song và bổ trợ cho nhau: Luồng xử lý và lập chỉ mục dữ liệu và luồng xử lý truy vấn và sinh câu trả lời.



Hình 9. Quy trình tổng thể của hệ thống

Đầu tiên, luồng xử lý và lập chỉ mục dữ liệu là một quy trình “ngoại tuyến” (offline), có nhiệm vụ xây dựng nên “bộ não” hay kho kiến thức cho trợ lý ảo. Quá trình này bắt đầu với việc tiếp nhận dữ liệu đầu vào là tập hợp các văn bản quy phạm pháp luật là các chỉ thị đã được thu thập. Hệ thống sử dụng thư viện **PyMuPDF** để mở từng tập tin PDF và trích xuất toàn bộ nội dung văn bản thô. Sau khi trích xuất, một quy trình làm sạch cơ bản được áp dụng để loại bỏ các yếu tố gây nhiễu như thông tin thừa ở đầu trang (headers), cuối trang (footers), và các khoảng trắng không cần thiết, đảm bảo dữ liệu văn bản là sạch và nhất quán. Do các mô hình ngôn ngữ có giới hạn về độ ngữ cảnh, văn bản sau khi làm sạch sẽ được phân chia thành các đoạn nhỏ hơn (chunks). Thay vì chia theo số lượng từ cố định, hệ thống áp dụng phương pháp phân đoạn theo ngữ nghĩa, dựa trên cấu trúc logic của văn bản quy phạm pháp luật như Chương, Điều, Khoản, giúp mỗi đoạn văn bản chứa một ý nghĩa trọn vẹn.

Sau khi phân đoạn, mỗi đoạn văn bản sẽ được đưa vào mô hình nhúng text-embedding-004 của Google để chuyển đổi thành một vector, tức mỗi chuỗi các con số mã hóa ngữ nghĩa của đoạn văn bản đó. Các vector nhúng này cùng với nội dung văn bản gốc tương ứng sẽ được lưu trữ và lập chỉ mục trong cơ sở dữ liệu vector **ChromaDB**, vốn được tối ưu cho việc tìm kiếm tương đồng với tốc độ cao. Khi quá trình này hoàn tất, kho kiến thức của trợ lý ảo đã được xây dựng xong và hoàn toàn sẵn sàng cho việc tra cứu.

Song song với quá trình chuẩn bị dữ liệu, luồng xử lý truy vấn “trực tuyến” (online) được kích hoạt mỗi khi người dùng đặt một câu hỏi thông qua giao diện ứng dụng. Câu hỏi này cũng được tiền xử lý cơ bản và sau đó đưa vào chính mô hình nhúng **text-embedding-004** đã dùng ở luồng trước để tạo ra một vector câu hỏi. Việc sử dụng cùng một mô hình là bắt buộc để đảm bảo vector của câu hỏi và vector của các đoạn văn bản nằm trong cùng một không gian ngữ nghĩa, cho phép việc so sánh diễn ra chính xác.

Vector của câu hỏi sau đó được sử dụng để thực hiện một cuộc tìm kiếm ngữ nghĩa (Semantic Search) trong cơ sở dữ liệu **ChromaDB**. Hệ thống sẽ tìm và trả về một số lượng k đoạn văn bản có vector gần với vector câu hỏi nhất, đây chính là những ngữ cảnh liên quan nhất để trả lời câu hỏi. Các đoạn văn bản này, cùng với câu hỏi gốc của người dùng, sẽ được kết hợp lại thành một câu lệnh (prompt) hoàn chỉnh và gửi đến mô hình ngôn ngữ lớn **Gemini**. Dựa trên prompt này, Gemini sẽ tổng hợp thông tin, suy luận và tạo ra một câu trả lời cuối cùng bằng ngôn ngữ tự nhiên. Cuối cùng, câu trả lời này được gửi trở lại giao diện ứng dụng và hiển thị cho người dùng, hoàn tất quy trình hỏi-đáp.

2.2. Cài đặt giải pháp

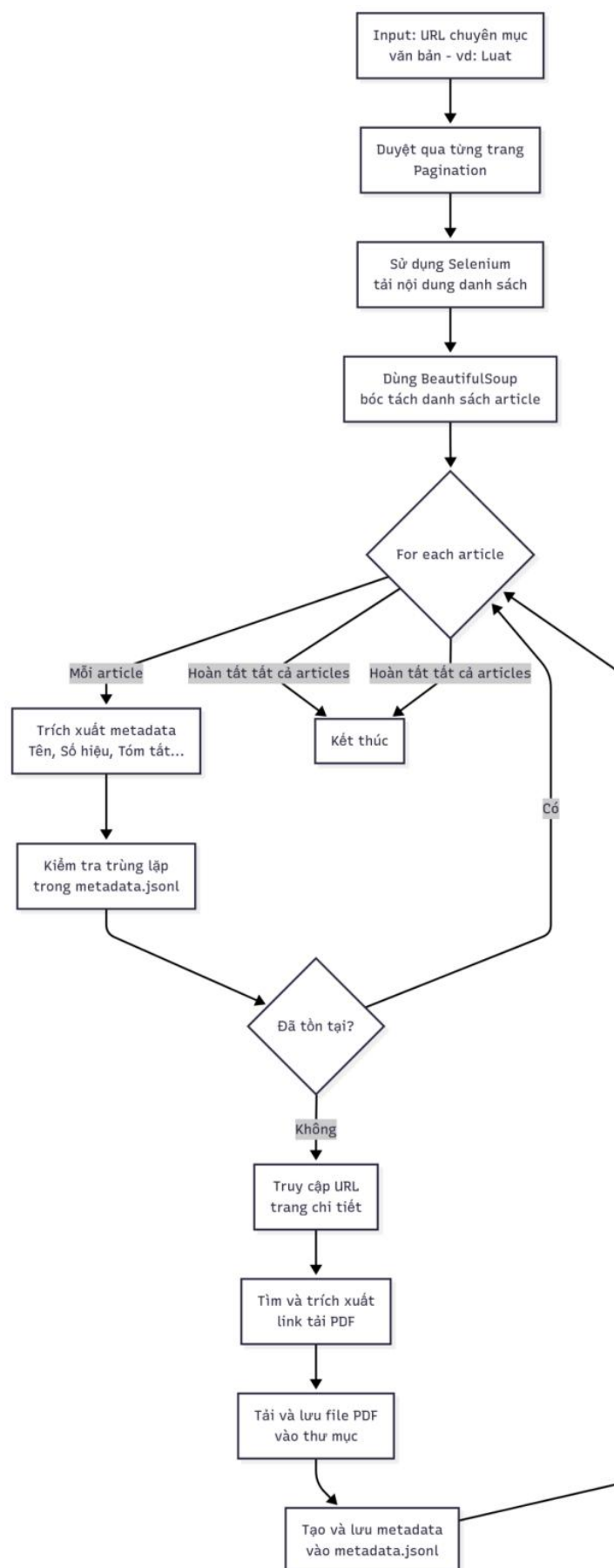
2.2.1. Thu thập dữ liệu



Hình 10. Trang Cổng Thông tin điện tử Chính phủ (Nguồn: <https://congbao.chinhphu.vn/>)

Nguồn dữ liệu được thu thập chủ yếu tại Cổng Thông tin điện tử Chính phủ, cụ thể là chuyên mục Công Báo (<https://congbao.chinhphu.vn/>). Đây là kênh thông tin điện tử chính thức của Chính phủ Việt Nam, đóng vai trò công khai các loại văn bản quy phạm pháp luật và văn bản hành chính do các cơ quan nhà nước ban hành. Trang Công Báo đảm bảo tính minh bạch và khả năng tiếp cận thông tin pháp luật cho người dân, doanh nghiệp và các tổ chức. Các văn bản đăng tải trên đây có độ tin cậy cao, phản ánh các quyết sách, chỉ đạo và quy định hiện hành của nhà nước. Một đặc điểm kỹ thuật quan trọng của nguồn này là hầu hết các văn bản được cung cấp dưới dạng tập tin PDF có chứa văn bản (text-based PDF), cho phép việc trích xuất nội dung text một cách trực tiếp và hiệu quả bằng các công cụ phần mềm.

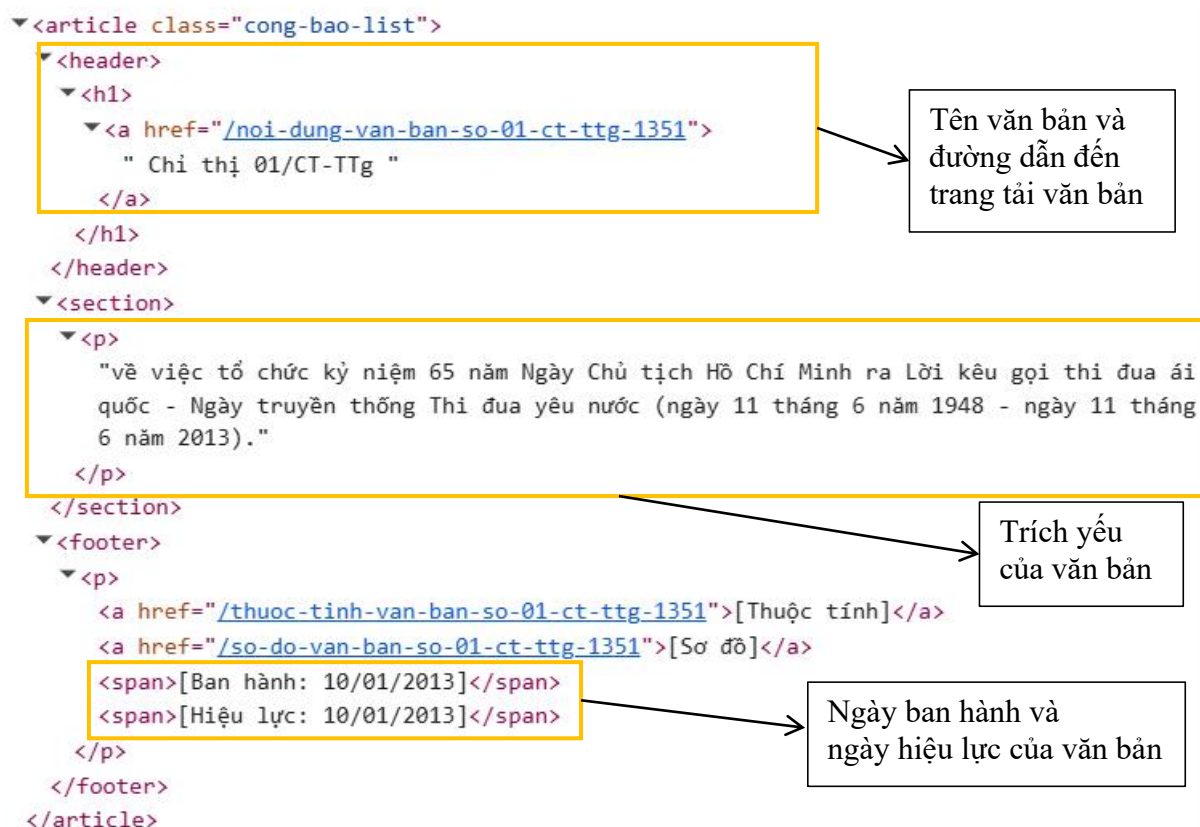
Từ Cổng Thông tin điện tử Chính phủ - Công Báo, tôi đã thu thập được 37 tập tin văn bản, là các văn bản thuộc loại “Chỉ thị”, tính đến ngày 01/07/2025. Hầu hết các tập tin này đều ở định dạng PDF text-based, thuận lợi cho quá trình xử lý và phân tích nội dung.



Hình 11. Quy trình cào dữ liệu

Trước khi tiến hành thu thập dữ liệu, một bước phân tích cấu trúc HTML của trang nguồn đã được thực hiện nhằm xác định các mẫu cấu trúc ổn định để bóc tách dữ liệu. Kết quả cho thấy trang web trình bày mỗi văn bản như một mục tin riêng biệt, đồng nhất và các cấu trúc mã HTML của các trang nguồn chứa các loại văn bản đều giống nhau. Mỗi mục tin được bọc hoàn toàn trong một thẻ HTML là `<article>` với thuộc tính `class="cong-bao-list"`. Thẻ `<article>` này đóng vai trò là một “container” chính, chứa toàn bộ các thông tin liên quan đến một văn bản cục thể như tiêu đề, tóm tắt và các siêu dữ liệu khác. Việc xác định container chung này là bước đầu tiên và quan trọng nhất để có thể duyệt qua và xử lý từng văn bản một cách độc lập.

Bên trong mỗi container `<article>`, thông tin được tổ chức một cách có cấu trúc và phân cấp rõ ràng. Cụ thể **tên văn bản** và **đường dẫn đến trang chi tiết** được tìm thấy trong thẻ `<a>` nằm bên trong thẻ `<h1>` của phần `<header>`. Phần **trích yếu** được đặt trong một thẻ `<p>` bên trong thẻ `<section>`. Cuối cùng, các thông tin về **ngày ban hành** và **ngày hiệu lực** được chứa trong thẻ `<span>` riêng biệt, nằm trong phần `<footer>` của `<article>`. Bằng cách nhận diện các quy tắc cấu trúc HTML cố định và có tính lặp lại này, việc lập trình để điều hướng một cách chính xác trong cây DOM và trích xuất từng mảng dữ liệu một cách đáng tin cậy.



Hình 12. Cấu trúc HTML của trang chứa loại văn bản “Chi thị”.

Trong hình là ví dụ mã nguồn của trang chứa chỉ thị 01/CT-TTg năm 2013.

Để xây dựng cơ sở dữ liệu cho hệ thống, một quy trình thu thập dữ liệu được thực hiện tự động từ Cổng Thông tin điện tử Chính phủ. Quy trình này được thực hiện bằng một

đoạn mã được lập trình bằng ngôn ngữ Python, kết hợp các thư viện chuyên dụng để đảm bảo hiệu suất và độ tin cậy.

Đầu tiên, một môi trường làm việc được thiết lập bằng thư viện **Selenium** cùng với **WebDriver Manager**, giúp tự động hóa việc điều khiển trình duyệt Chrome. Trình duyệt được cấu hình để hoạt động ở chế độ nền (headless), cho phép đoạn mã chạy hiệu quả mà không cần hiển thị giao diện đồ họa. Hai thực thể (instance) trình duyệt riêng biệt được khởi tạo đảm nhận các nhiệm vụ khác nhau: **driver_1** đảm nhận việc duyệt qua các trang danh sách văn bản, trong khi **driver_2** được dành riêng để truy cập vào từng trang chi tiết của mỗi văn bản. Cách này giúp quản lý các luồng điều hướng một cách độc lập và hiệu quả.

Quy trình chính được thực hiện trong một vòng lặp, tự động duyệt qua tất cả các trang của một chuyên mục văn bản cụ thể. Tại mỗi trang danh sách, **driver_1** sẽ tải về toàn bộ mã nguồn HTML. Sau đó, mã nguồn này được đưa vào thư viện **BeautifulSoup** để tiến hành phân tích và tạo ra một cây cấu trúc dữ liệu. Dựa trên cấu trúc này, tôi đã lập trình để tìm kiếm và trích xuất tất cả các thẻ `<article>` có thuộc tính `class="cong-bao-list"`, mỗi thẻ này tương ứng với thông tin của một văn bản.

Bên trong vòng lặp duyệt qua từng mục `<article>` đã tìm thấy, một quy trình bóc tách và chuẩn hóa dữ liệu chi tiết được thực hiện ở cấp độ mã nguồn thông qua **driver_2**. Mỗi mục được lưu vào biến **item**. Để lấy **Tên văn bản**, tôi đã sử dụng phương thức `item.find("header h1 a").get_text(strip=True)` để điều hướng chính xác qua cây cấu trúc HTML và trích xuất nội dung text thô. Ngay sau đó, biểu thức chính quy `r"\s+"` được áp dụng thông qua hàm `re.sub()` để thay thế một hoặc nhiều ký tự khoảng trắng liên tiếp bằng một khoảng trắng duy nhất, nhằm chuẩn hóa và làm sạch tiêu đề. Đồng thời, **đường dẫn** (URL) của trang chi tiết được lấy ra bằng cách truy cập trực tiếp vào thuộc tính `href` của thẻ `<a>`.

Tuy nhiên, các trường dữ liệu khác đòi hỏi xử lý phức tạp hơn. Do **Số hiệu** của mỗi văn bản nằm trong trường **Tên văn bản** được lưu trước đó, nên tôi đã xây dựng hàm `get_so_hieu()` để tách **Số hiệu** ra khỏi chuỗi tên văn bản. Hàm này sử dụng biểu thức chính quy `r"\d"` để thực thi lệnh `re.search()`, với mục đích tìm kiếm vị trí chỉ mục (index) của ký tự số đầu tiên xuất hiện trong chuỗi. Do số hiệu văn bản luôn bắt đầu là một con số, hàm sẽ thực hiện cắt chuỗi (string slicing) từ chỉ số đầu tiên đó cho đến hết chuỗi, trả về phần số hiệu đã được tách riêng một cách chính xác. Ví dụ, với tên văn bản là “Chỉ thị 01/CT-TTg” thì số hiệu của văn bản sau khi đưa vào hàm `get_so_hieu()` sẽ là “01/CT-TTg”. Tương tự, để xử lý **Ngày ban hành** và **Ngày hiệu lực**, đoạn mã trước hết sẽ tìm tất cả thẻ `<span>` trong phần `<footer>` của mục tin, sau đó lặp qua từng thẻ và kiểm tra nội dung text của nó có chứa từ khóa “ban hành” hay “hiệu lực” hay không. Khi đã xác định đúng thẻ, nội dung text thô sẽ được đưa vào hàm `get_ngaybh_ngayhl()`. Hàm này cũng áp dụng biểu thức `r"\d"` để xác định điểm bắt đầu của chuỗi ngày tháng, sau đó thực hiện cắt chuỗi đến ký tự áp chót nhằm loại bỏ các văn bản mô tả ở phía trước và các ký tự không cần thiết ở phía sau. Ví dụ như trong hình , có hai thẻ `<span>` trong phần `<footer>`, kết quả sau khi đưa nội dung text thô vào hàm `get_ngaybh_ngayhl()` thì ngày ban hành sẽ là “10/01/2013” và ngày hiệu lực là “10/01/2013”.

```
<ul class="dropdown-menu dropdown-menu-right">
  <li>
    <a href="/tai-ve-van-ban-so-01-ct-ttg-1351-442?format=pdf" target="_blank">2013_53 + 54-01_CT-TTg.pdf</a>
  </li>
</ul>
```

Hình 13. Mã nguồn trang chi tiết chứa đường dẫn tải văn bản trong thẻ `<ul>`

Tại trang chi tiết của văn bản, tiếp tục phân tích cấu trúc HTML để xác định vị trí của liên kết tải về. Cụ thể, nó sẽ tìm đến thẻ `<ul>` có thuộc tính `class="dropdown-menu"` và duyệt qua các thẻ con `<li>` bên trong. Sau đó, đoạn mã sẽ bắt đầu xem các thẻ `<a>` trong mỗi thẻ `<li>` để tìm ra thẻ có thuộc tính `href` chứa chuỗi `"format=pdf"`. Khi đã có được URL chính xác của tập tin PDF, hàm `download_pdf()` sẽ được gọi. Hàm này được cài đặt để sử dụng thư viện **requests** với tham số **stream=True**, cho phép tải các tập tin có dung lượng lớn một cách hiệu quả bằng cách đọc dữ liệu theo từng khối nhỏ (chunk) thay vì tải toàn bộ về bộ nhớ cùng lúc. Tập tin PDF sau đó được lưu vào thư mục cục bộ đã được cấu hình trước.

Song song với việc lưu tập tin PDF, một từ điển (dictionary) chứa toàn bộ siêu dữ liệu của văn bản được tạo ra. Từ điển này sau đó được chuyển cho hàm `append_metadata()` để ghi vào tệp `metadata_congthongtindientu_chithi.jsonl`. Hàm này mở tệp ở chế độ ghi nối tiếp với mã hóa utf-8 và sử dụng lệnh `json.dump()` để chuyển đổi từ điển thành một chuỗi JSON. Mỗi bản ghi được ghi trên một dòng riêng biệt, theo đúng chuẩn định dạng JSON Lines. Cách lưu trữ này không chỉ đảm bảo tính toàn vẹn của dữ liệu mà còn cực kỳ hiệu quả khi cần bổ sung thông tin mới vào một tệp lớn mà không cần phải đọc và ghi lại toàn bộ tệp.

Mỗi tập tin metadata lưu mỗi loại văn bản với các trường dữ liệu sau:

- Tên tập tin: Tên của tập tin văn bản đã tải. Vì tên văn bản và tên tập tin không giống nhau nên sẽ lưu trữ tên tập tin để thuận tiện cho việc tìm kiếm.
- Tên văn bản: Tên của văn bản đã tải.
- Số hiệu: Số hiệu của văn bản sau khi đã xử lý.
- Ngày ban hành: Ngày ban hành của văn bản.
- Ngày hiệu lực: Ngày có hiệu lực chính thức của văn bản.
- Loại văn bản: Loại của văn bản. Vì các văn bản được xử lý có ba loại khác nhau là chỉ thị, công điện, luật nên việc lưu loại văn bản sẽ giúp chương trình biết được đang xử lý loại văn bản nào và thực hiện quy trình gì.
- Nguồn: Nơi tải văn bản.
- Trạng thái tải: "Tải thành công" và "Không tải được file". Trường hợp "Không tải được file" là do trong quá trình cào dữ liệu có thể đường truyền không ổn định ảnh hưởng đến việc tải văn bản nên nếu trường hợp này xảy ra. Tôi sẽ tải văn bản này thủ công để thu được toàn bộ phần văn bản cần thu thập.

Cuối cùng, sau khi hoàn thành quá trình thu thập dữ liệu, tôi đã thu thập tổng cộng 36 tập tin thuộc loại văn bản "Chỉ thị".

2.2.2. Tiền xử lý dữ liệu

2.2.2.1. Trích xuất văn bản thô

Bước đầu tiên là chuyển đổi các tài liệu PDF từ định dạng trình bày trực quan sang một cấu trúc dữ liệu mà máy tính có thể phân tích được. Thách thức ở giai đoạn này không chỉ là lấy được nội dung chữ, mà còn phải giữ lại được các thông tin định dạng ẩn, vốn là chìa khóa để hiểu được logic của văn bản pháp luật.

Hệ thống sử dụng thư viện **PyMuPDF** (fitz), một công cụ mạnh mẽ và hiệu suất cao cho việc xử lý PDF trong Python. Quy trình không chỉ đơn thuần “đọc” toàn bộ văn bản một cách tuần tự. Thay vào đó, nó thực hiện một cuộc “giải phẫu” chi tiết tập tin PDF bằng cách sử dụng phương thức `page.get_text(“dict”)`. Phương thức này trả về một cấu trúc dữ liệu dạng cây, cho phép chương trình duyệt qua từng trang, sau đó đi sâu vào từng khối văn bản (block), từng dòng (line), và cuối cùng là từng đoạn chữ có cùng định dạng (span). Tại cấp độ span, hệ thống sẽ trích xuất một cách tỉ mỉ không chỉ nội dung văn bản mà còn cả một từ điển các thuộc tính định dạng quan trọng như:

- Font: Tên font chữ.
- Size: Kích cỡ chữ.
- Flags: Các cờ định dạng như in đậm, in nghiêng.

Để minh họa rõ hơn, hãy xem ví dụ trong hình 18. Hình 18a là một đoạn trích từ file PDF gốc, trong đó tiêu đề “**I. ĐỐI VỚI CHỦ ĐẦU TƯ, BÊN MỜI THẦU**” được định dạng in đậm và có kích thước lớn hơn. Hình 18b là kết quả dữ liệu có cấu trúc đã trích xuất được từ đoạn văn bản đó.

I. ĐỐI VỚI CHỦ ĐẦU TƯ, BÊN MỜI THẦU

1. Thực hiện nghiêm túc các trách nhiệm của mình, bảo đảm tổ chức lựa chọn nhà thầu hiệu quả, nhanh chóng theo đúng quy định. Cụ thể là:

(a) Đoạn văn bản gốc trong tập tin PDF

```
{'text': 'I. ĐỐI VỚI CHỦ ĐẦU TƯ, BÊN MỜI THẦU', 'font': 'TimesNewRomanPS-BoldMT', 'size': 14.039999961853027, 'flags': 20, 'bbox': (86.75863647460938, 231.81158447265625, 364.34771728515625, 247.35386657714844), 'page': 1}
{'text': '1. Thực hiện nghiêm túc các trách nhiệm của mình, bảo đảm tổ chức lựa chọn', 'font': 'TimesNewRomanPSMT', 'size': 14.039999961853027, 'flags': 4, 'bbox': (86.76007080078125, 255.3314208984375, 534.8692626953125, 270.87371826171875), 'page': 1}
{'text': 'nhà thầu hiệu quả, nhanh chóng theo đúng quy định. Cụ thể là:', 'font': 'TimesNewRomanPSMT', 'size': 14.039999961853027, 'flags': 4, 'bbox': (64.07992553710938, 274.8919677734375, 419.3285217285156, 290.43426513671875), 'page': 1}
```

(b) Dữ liệu có cấu trúc được truy xuất

Hình 14. Minh họa kết quả của bước trích xuất chi tiết

Như kết quả trong hình 14b, có thể thấy hệ thống không chỉ lấy nội dung một cách riêng biệt. Quan trọng hơn, nó đã nhận diện và ghi lại chính xác tên *font* chữ cho từng phần: “‘font’: ‘TimesNewRomanPS-BoldMT’” cho tiêu đề mục **I** (cho biết đây là chữ in đậm), phân biệt rõ ràng với *font* chữ của đoạn văn bản thường là ‘TimesNewRomanPSMT’. Kết quả chi tiết và giàu thông tin này chính là đầu vào cực kỳ quan trọng, tạo tiền đề cho các thuật toán ở bước sau có thể dựa vào các dấu hiệu định dạng này để phân tích cấu trúc văn bản một cách chính xác.

2.2.2.2. Làm sạch văn bản

Sau khi được trích xuất, dữ liệu văn bản vẫn còn ở dạng thô và chứa rất nhiều “nhiều” - là những thông tin không thuộc về nội dung pháp lý cốt lõi. Để giải quyết vấn đề này, một module làm sạch văn bản chuyên dụng đã được thiết kế và cài đặt, với nhiệm vụ chính là loại bỏ các thành phần không cần thiết và chuẩn hóa định dạng văn bản.



(a) Phần đầu trang (header) của văn bản



(b) Phần cuối trang (footer) của một số văn bản

Hình 15. Hình 15a và 15b là các thông tin gây nhiễu, không cần lưu trữ

Giải pháp làm sạch được xây dựng dựa trên sức mạnh của biểu thức chính quy (regular expressions - regex), một công cụ cho phép định nghĩa các mẫu văn bản phức tạp để tìm kiếm và xử lý chuỗi. Thiết kế của module này tuân theo nguyên tắc module hóa, trong đó toàn bộ các mẫu biểu thức chính quy được định nghĩa tập trung tại một **kho lưu trữ mẫu (pattern repository)**. Cách tiếp cận này giúp việc quản lý, cập nhật và tinh chỉnh các quy tắc lọc trở nên dễ dàng mà không cần thay đổi logic xử lý chính. Các kho lưu trữ mẫu bao gồm:

- **HEADER_FOOTER_PATTERNS:** Là danh sách lưu trữ các mẫu biểu thức chính quy được thiết kế để nhận diện và loại bỏ các thông tin thường lặp lại ở đầu trang (header) và chân trang (footer) của tài liệu Công báo. Các mẫu này chủ yếu nhắm đến các chuỗi ký tự như "CÔNG BÁO/Số...", số trang đơn thuần, hoặc các định dạng ngày tháng đi kèm với số hiệu Công báo, vốn là những thông tin nhiễu không thuộc về nội dung pháp lý.
- **SIGNATURE_PATTERNS:** Bộ mẫu này có nhiệm vụ xác định điểm kết thúc của nội dung văn bản. Nó bao gồm các mẫu để nhận diện khối chữ ký, bao gồm

các chức danh ("THỦ TƯỚNG", "BỘ TRƯỞNG"), các từ viết tắt của chức danh ("KT.", "TM."), tên của các lãnh đạo thường ký văn bản, và ký hiệu kết thúc văn bản ("./."). Việc nhận diện chính xác khối chữ ký giúp hệ thống cắt bỏ toàn bộ phần nội dung không liên quan ở phía sau.

- **IGNORE_PATTERNS:** Đây là một danh sách toàn diện, chứa các mẫu để loại bỏ hoàn toàn các dòng văn bản mang tính khuôn mẫu, không có giá trị ngữ nghĩa. Danh sách này bao gồm các mẫu cho khẩu hiệu quốc gia ("CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM"), thông tin nhà xuất bản ("VĂN PHÒNG CHÍNH PHỦ XUẤT BẢN"), địa chỉ, email, thông tin ký điện tử và các dòng trống, đảm bảo chỉ giữ lại phần nội dung pháp lý cốt lõi.
- **SECTION_PATTERNS:** Đây là bộ mẫu quan trọng nhất, đóng vai trò "xương sống" trong việc bóc tách cấu trúc phân cấp của văn bản. Nó bao gồm các mẫu riêng biệt để nhận diện từng cấp bậc: chương (dựa trên chữ số La Mã), mục (dựa trên chữ số Ả Rập), khoản (dựa trên chữ cái thường và dấu ngoặc đơn), và diem (dựa trên dấu gạch ngang đầu dòng).
- **METADATA_PATTERNS:** Bộ mẫu này được dùng để tìm và trích xuất các thông tin định danh quan trọng của văn bản, chẳng hạn như so_hieu (tìm chuỗi ký tự sau từ khóa "Số:"), date (tìm ngày ban hành của văn bản), và type (tìm loại văn bản như "CHỈ THỊ" ở đầu tài liệu).

Module này hoạt động như một bộ lọc đa lớp, xử lý văn bản theo từng bước cụ thể:

1. **Loại bỏ các dòng thông tin chung:** Đầu tiên, sử dụng một bộ mẫu từ kho lưu trữ để nhận diện và loại bỏ hoàn toàn các dòng văn bản chứa thông tin khuôn mẫu như khẩu hiệu quốc gia, thông tin nhà xuất bản, địa chỉ email, và các thông tin không liên quan khác.
2. **Loại bỏ đầu trang và chân trang:** Tiếp theo, một bộ mẫu khác được áp dụng để xử lý các thông tin lặp lại ở đầu và cuối mỗi trang, thường chứa số hiệu Công báo và số trang.
3. **Xác định và cắt bỏ phần cuối văn bản:** Quy trình tiếp theo là xác định điểm kết thúc của nội dung pháp lý. Cụ thể là tìm kiếm các dấu hiệu như khối chữ ký của người ban hành hoặc ký hiệu kết thúc văn bản chính thức ("./."). Toàn bộ nội dung xuất hiện sau các dấu hiệu này sẽ bị cắt bỏ để đảm bảo không đưa vào các thông tin phụ lục hoặc trang trắng không cần thiết.
4. **Chuẩn hóa khoảng trắng:** Cuối cùng, sau khi đã loại bỏ các thành phần nhiễu, toàn bộ văn bản còn lại sẽ được đi qua một bước chuẩn hóa để thay thế các chuỗi khoảng trắng hoặc ký tự xuống dòng liên tiếp bằng một ký tự duy nhất, đảm bảo dữ liệu đầu ra có định dạng đồng nhất.

2.2.2.3. Bóc tách siêu dữ liệu (metadata):

Sau khi văn bản đã được làm sạch, bước tiếp theo là bóc tách các thông tin định danh quan trọng, hay còn gọi là siêu dữ liệu (metadata). Các thông tin này (như số hiệu, ngày ban hành,...) là cực kỳ cần thiết cho việc quản lý, trích dẫn và sau này là hiển thị cho người dùng cuối. Để thực hiện nhiệm vụ này, một module trích xuất siêu dữ liệu chuyên dụng đã được thiết kế, hoạt động dựa trên sự kết hợp giữa các biểu thức chính quy và các quy tắc logic suy luận.

Module này được thiết kế để thực hiện một quy trình bóc tách thông minh, chia thành nhiều giai đoạn để xử lý các loại thông tin khác nhau:

- 1. Trích xuất thông tin định danh cơ bản:** Đầu tiên, module sẽ duyệt qua phần đầu của văn bản đã được làm sạch để tìm kiếm các thông tin cơ bản nhất. Nó sử dụng các mẫu biểu thức chính quy định nghĩa trong METADATA_PATTERNS để nhận diện:
 - Loại văn bản: Bằng cách tìm kiếm chuỗi ký tự in hoa và đứng độc lập như “CHỈ THỊ”.
 - Số hiệu: Bằng cách tìm kiếm mẫu “Số:” theo sau là một chuỗi ký tự định danh (ví dụ: 28/CT-TTg).
 - Nơi ban hành: Bằng cách phân tích các ký tự viết tắt trong “Số hiệu” (ví dụ “CT-TTg” được ánh xạ tới “THỦ TƯỚNG CHÍNH PHỦ”), hệ thống có thể điền thông tin này một cách chính xác mà không cần tìm kiếm trực tiếp trong văn bản.
- 2. Tra cứu và bổ sung từ nguồn dữ liệu ngoài:** Để đảm bảo chính xác và đầy đủ, các thông tin như Ngày ban hành, Ngày hiệu lực và Trích yếu không được trích xuất trực tiếp từ nội dung PDF do có thể định dạng không nhất quán). Thay vào đó, hệ thống sử dụng lại các thông tin metadata sau khi hoàn thành bước thu thập dữ liệu. Cụ thể, nó sử dụng “Số hiệu” đã trích xuất được làm khóa chính để tìm kiếm và lấy về các thông tin tương ứng.
- 3. Phân tích khối chữ ký:** Để tối ưu và tăng độ chính xác, module chỉ tập trung quét **50 khối văn bản cuối cùng** của tài liệu. Quy trình quét được thực hiện theo thứ tự **từ dưới lên trên**:
 - Đầu tiên, nó tìm kiếm một dòng khớp với mẫu chức danh (ví dụ: "THỦ TƯỚNG", "BỘ TRƯỞNG").
 - Sau đó, nó tiếp tục tìm kiếm một dòng bên dưới khớp với mẫu định dạng tên người Việt Nam.

```
"so_hieu": "1617/CT-TTg",
"loai_van_ban": "CHỈ THỊ",
"noi_ban_hanh": "THỦ TƯỚNG CHÍNH PHỦ",
"ngay_ban_hanh": "19/09/2011",
"ngay_hieu_luc": "19/09/2011",
"ten_van_ban": "Chỉ Thị 1617/CT-TTg",
"trich_yeu": "Về việc tăng cường thực hiện và chấn chỉnh công tác quản lý đầu tư trực tiếp nước ngoài trong thời gian tới",
"nguoi_ky": "Nguyễn Tấn Dũng",
"chuc_vu_nguoi_ky": "THỦ TƯỚNG",
"file_name": "2011_515+_516-1617_CT-TTg.pdf",
```

Hình 16. Các trường và nội dung của các siêu dữ liệu sau khi bóc tách metadata

2.2.2.4. Chia đoạn văn bản

Sau khi văn bản đã được làm sạch, hệ thống phải đối mặt với một thách thức cốt lõi: làm thế nào để chia một tài liệu pháp lý thành những đơn vị thông tin nhỏ hơn, dễ quản lý hơn. Đây là một yêu cầu bắt buộc, không chỉ vì các mô hình ngôn ngữ lớn (LLMs) có giới hạn về độ dài văn bản đầu vào, mà còn vì chất lượng của toàn bộ hệ thống RAG phụ thuộc gần như hoàn toàn vào chất lượng của các đoạn văn bản (chunks) được truy xuất.

Mục tiêu của bước này không phải là chia nhỏ văn bản một cách cơ học (ví dụ: cắt sau mỗi 500 từ). Một cách tiếp cận như vậy sẽ phá vỡ cấu trúc ngữ nghĩa của văn bản, có thể tách rời một tiêu đề khỏi nội dung của nó, hoặc cắt đôi một điều luật quan trọng, làm cho ngữ cảnh được truy xuất trở nên vô nghĩa. Thay vào đó, mục tiêu sẽ là thực hiện **Phân đoạn theo ngữ nghĩa (Semantic Chunking)**. Tức là, chia văn bản thành các đơn vị thông tin có ý nghĩa trọn vẹn, bám sát theo đúng cấu trúc phân cấp logic vốn có của một văn bản pháp luật (Chương, Điều, Khoản, v.v.). Một "chunk" lý tưởng phải là một đơn vị tự chứa, đủ thông tin để có thể hiểu được mà không cần các phần xung quanh.

Ví dụ với đoạn văn bản sau:

1. Bộ trưởng, Thủ trưởng cơ quan ngang Bộ có trách nhiệm:

a) Tổ chức thực hiện nghiêm Quyết định số 1573/QĐ-TTg ngày 11 tháng 9 năm 2015 của Thủ tướng Chính phủ ban hành kế hoạch triển khai thi hành Luật ban hành văn bản quy phạm pháp luật trong phạm vi ngành, lĩnh vực được phân công; quán triệt nội dung và tinh thần các quy định của Luật bằng các hình thức, biện pháp phù hợp đối với cán bộ, công chức trong Bộ, ngành;

b) Xác định công tác xây dựng, ban hành văn bản quy phạm pháp luật là một trong các nhiệm vụ quan trọng hàng đầu trong chương trình công tác của Bộ, ngành mình, từ đó tập trung thích đáng các nguồn lực để nâng cao chất lượng đề nghị xây dựng văn bản quy phạm pháp luật; chất lượng soạn thảo, ban hành văn bản quy phạm pháp luật;

Hình 17. Mục 1, khoản a, b của chỉ thị 28/CT-TTg năm 2015

Sau khi hoàn thành bước này, cấu trúc cây **đúng** được thể hiện như bên dưới:

```
"muc": [  
  {  
    "ten_muc": "Mục I",  
    "tiêu_de": "Bộ trưởng, Thủ trưởng cơ quan ngang Bộ có trách nhiệm:",  
    "khoan": [  
      {  
        "ten_khoan": "Khoản a",  
        "tiêu_de": "Tổ chức thực hiện nghiêm Quyết định số 1573/QĐ-TTg ngày 11  
tháng 9 năm 2015 của Thủ tướng Chính phủ ban hành kế hoạch triển khai thi hành  
Luật ban hành văn bản quy phạm pháp luật trong phạm vi ngành, lĩnh vực được phân  
công; quán triệt nội dung và tinh thần các quy định của Luật bằng các hình thức,  
biện pháp phù hợp đối với cán bộ, công chức trong Bộ, ngành;",  
        "diem": []  
      },  
      {  
        "ten_khoan": "Khoản b",  
        "tiêu_de": "Xác định công tác xây dựng, ban hành văn bản quy phạm pháp  
luật là một trong các nhiệm vụ quan trọng hàng đầu trong chương trình công tác của  
Bộ, ngành mình, từ đó tập trung thích đáng các nguồn lực để nâng cao chất lượng đề  
nghị xây dựng văn bản quy phạm pháp luật; chất lượng soạn thảo, ban hành văn bản  
quy phạm pháp luật;",  
        "diem": []  
      },  
      .....  
    ]  
  }  
]
```

Để thực hiện việc phân đoạn theo ngữ nghĩa, một thuật toán phân tích cấu trúc có trạng thái đã được thiết kế. Thuật toán này hoạt động như một người đọc thông minh, duyệt qua văn bản sạch từ trên xuống dưới và luôn “ghi nhớ” mình đang ở đâu trong cấu trúc văn bản. Quy trình phân tích hoạt động như sau:

1. **Xác định điểm bắt đầu nội dung:** Đầu tiên, thuật toán sẽ bỏ qua các nội dung chung của văn bản và chỉ bắt đầu quá trình phân tích khi gặp dấu hiệu cấu trúc đầu tiên (ví dụ: “Chương I.” hoặc “1.”).
2. **Đối chiếu mẫu theo thứ tự ưu tiên:** Tại mỗi dòng văn bản, hệ thống sẽ kiểm tra theo thứ tự ưu tiên từ cao xuống thấp: dòng này có phải là tiêu đề “Chương” không? Nếu không, có phải là tiêu đề “Mục” không? Cứ tiếp tục như vậy cho đến “Khoản” và “Điểm”.
3. **Xây dựng cây cấu trúc và quản lý trạng thái:**
 - Khi một tiêu đề "**Chương**" được nhận diện, một đối tượng "Chương" mới sẽ được khởi tạo trong cấu trúc dữ liệu JSON đầu ra. Trạng thái của thuật toán lúc này được cập nhật: "đang ở trong một Chương mới".
 - Khi một tiêu đề "**Mục**" được tìm thấy sau đó, một đối tượng "Mục" sẽ được tạo và thêm vào như một "nút con" của đối tượng "Chương" hiện tại. Trạng thái được cập nhật: "đang ở trong Mục x của Chương y".

- Logic này tiếp tục lồng vào nhau một cách đệ quy cho "Khoản" và "Điều", tạo ra một cây cấu trúc dữ liệu phản ánh chính xác hệ thống phân cấp của văn bản gốc.
4. **Thu thập tiêu đề đa dòng:** Khi một mẫu tiêu đề (như "1.") được nhận diện, hệ thống sẽ tiếp tục đọc các dòng tiếp theo và nối chúng lại cho đến khi gặp một dòng trống hoặc một mẫu cấu trúc mới, đảm bảo thu thập được toàn bộ nội dung của tiêu đề.
 5. **Gán nội dung:** Sau khi một mẫu tiêu đề cấu trúc (ví dụ: "Chương I.") được nhận diện, thuật toán sẽ tiếp tục đọc và nối các dòng văn bản liền kề vào tiêu đề đó cho đến khi gặp một dòng trống hoặc một mẫu cấu trúc mới. Khi tiêu đề đã được thu thập hoàn chỉnh, bất kỳ dòng nào sau đó không khớp với một mẫu tiêu đề mới sẽ được xác định là nội dung. Toàn bộ phần nội dung này sẽ được gán cho thành phần cấu trúc gần nhất vừa được xác định (ví dụ, nội dung sẽ thuộc về 'Khoản a' cho đến khi 'Khoản b' xuất hiện). Quy trình kép này đảm bảo cả tiêu đề và nội dung tương ứng của nó luôn được liên kết chính xác với nhau trong cây cấu trúc dữ liệu cuối cùng.

Kết quả của toàn bộ quá trình phức tạp này là một cấu trúc dữ liệu JSON có tổ chức chặt chẽ, là một bản sao số hóa gần như hoàn hảo của cấu trúc logic trong tài liệu PDF gốc. Dữ liệu có cấu trúc này là nền tảng vàng cho các bước tiếp theo. Từ đây, hệ thống có thể dễ dàng tạo ra các "chunk" chất lượng cao để đưa vào cơ sở dữ liệu vector. Ví dụ, mỗi "chunk" có thể là nội dung của một "Khoản" hoặc một "Điều" cụ thể, đi kèm với toàn bộ "đường dẫn" ngữ cảnh của nó (ví dụ: "Chương I -> Mục 2 -> Khoản a"). Điều này đảm bảo khi người dùng đặt câu hỏi, hệ thống có thể truy xuất đến đúng đơn vị thông tin nhỏ nhất và chính xác nhất.

2.2.2.5. Chuyển đổi cây cấu trúc thành các chunk ngữ nghĩa:

Sau khi đã tái tạo thành công cấu trúc phân cấp của văn bản pháp luật dưới dạng cây JSON, bước tiếp theo là "làm phẳng" (flatten) cấu trúc này thành một danh sách các đơn vị thông tin riêng lẻ, hay còn gọi là các "chunk". Đây là bước chuẩn bị cuối cùng và quan trọng nhất trước khi đưa dữ liệu vào quá trình nhúng vector, với mục tiêu tạo ra các chunk không chỉ nhỏ gọn mà còn phải **giàu ngữ nghĩa** và **tự chứa (self-contained)**. Một chunk lý tưởng phải chứa đủ thông tin để khi được truy xuất, nó có thể cung cấp một ngữ cảnh đầy đủ cho Mô hình Ngôn ngữ Lớn mà không cần phải tham chiếu đến các chunk khác.

Để đạt được mục tiêu trên, một quy trình gồm hai giai đoạn chính đã được thiết kế và cài đặt: (1) Tạo các chunk ngữ nghĩa ban đầu bằng một thuật toán duyệt cây đệ quy, và (2) Tinh chỉnh kích thước của các chunk này bằng một phương pháp hybrid.

Giai đoạn 1: Tạo chunk ngữ nghĩa bằng thuật toán đệ quy

Giai đoạn này sử dụng một **thuật toán duyệt cây đệ quy** để đi sâu vào từng nhánh của cấu trúc JSON đã được tạo ở bước trước. Logic hoạt động của thuật toán này như sau:

1. **Khởi tạo và duyệt cây:** Thuật toán bắt đầu từ cấp cao nhất của mỗi văn bản (danh sách các "Chương"). Nó sẽ đi sâu vào từng nút (node) trong cây, từ "Chương" xuống "Mục", "Khoản", và cuối cùng là "Điểm".
2. **Tích lũy ngữ cảnh và đường dẫn trích dẫn:** Trong quá trình duyệt, thuật toán duy trì hai danh sách quan trọng:
 - Context_path (Đường dẫn ngữ cảnh): Đây là một danh sách chứa các **tiêu đề** của tất cả các nút cha mà nó đã đi qua. Ví dụ, khi thuật toán đang xử lý một "Khoản", danh sách này sẽ chứa tiêu đề của Chương và Mục chứa Khoản đó.
 - Citation_path (Đường dẫn trích dẫn): Tương tự, danh sách này chứa các **định danh** (như "Chương I", "Mục 2", "Khoản a") của các nút cha.
3. **Tạo chunk tại nút lá:** Thuật toán chỉ thực sự tạo ra một "chunk" khi nó đi đến một "nút lá" của cây—tức là một thành phần không có cấp con nữa. Trong cấu trúc văn bản pháp luật, nút lá thường là một "Điểm" (nội dung bắt đầu bằng dấu gạch ngang) hoặc một "Khoản"/"Mục" không chứa các cấp con bên trong.
4. **Làm giàu chunk:** Đây là bước cốt lõi. Thay vì chỉ lấy nội dung của nút lá, thuật toán sẽ tạo ra một đoạn văn bản mới được làm giàu một cách toàn diện bằng cách kết hợp:
 - Siêu dữ liệu quan trọng: Tên văn bản, ngày ban hành, người ký và chức vụ người ký.
 - Đường dẫn ngữ nghĩa đầy đủ: Toàn bộ các tiêu đề trong context_path được nối lại với nhau.
 - Nội dung chính: Nội dung của chính nút lá đó.

Ví dụ, một chunk được tạo ra từ "Khoản a" của Mục 1, Chương I trong Chỉ thị 1722/CT-TTg sẽ có nội dung dạng:

```
{
  "id": "a7a32fd1-77c1-4e6d-9a53-449d9aa6ea5c",
  "source": "Chỉ Thị 1722/CT-TTg",
  "location": "Chương I, Mục 1, Khoản a",
  "content": "Văn bản: Chỉ Thị 1722/CT-TTg. Ngày ban hành: 17/09/2010. Người ký: Nguyễn Tấn Dũng, THỦ TƯỚNG. Nội dung: Chỉ Thị 1722/CT-TTg. VỀ TỔ CHỨC TẬP HUẤN VÀ TUYÊN TRUYỀN, PHỔ BIẾN NGHỊ ĐỊNH SỐ 63/2010/NĐ-CP NGÀY 08 THÁNG 6 NĂM 2010 VÀ NGHỊ ĐỊNH SỐ 20/2010/NĐ-CP NGÀY 14 THÁNG 02 NĂM 2010. Giao Văn phòng Chính phủ. Biên soạn ..... tuân thủ thấp nhất.",
  "issue_date": "17/09/2010",
  "effective_date": "17/09/2010",
  "signatory": "Nguyễn Tấn Dũng",
  "signatory_position": "THỦ TƯỚNG"
}
```

Giai đoạn 2: Tinh chỉnh kích thước chunk bằng phương pháp Hybrid

Một số chunk được tạo ra ở giai đoạn 1 có thể quá dài. Để xử lý, một quy trình tinh chỉnh kích thước hybrid được áp dụng:

1. **Kiểm tra Ngưỡng độ dài:** Hệ thống kiểm tra độ dài của mỗi chunk. Nếu độ dài nhỏ hơn ngưỡng an toàn (ví dụ: 1500 ký tự), chunk đó được giữ nguyên.
2. **Phân đoạn theo Ký tự:** Nếu một chunk vượt quá ngưỡng, nó sẽ được đưa qua `RecursiveCharacterTextSplitter`. Công cụ này sẽ chia nhỏ đoạn văn bản dựa trên các ký tự ngắt câu tự nhiên (như dấu chấm, xuống dòng).
3. **Phân đoạn theo Ngữ nghĩa:** Trong trường hợp một sub-chunk sau khi chia theo ký tự vẫn còn quá dài, nó sẽ được đưa vào `SemanticChunker` của `LangChain`. Công cụ này sử dụng chính mô hình embedding để tìm ra các "điểm gãy" về mặt ngữ nghĩa và thực hiện việc chia nhỏ tại các điểm đó.

Kết quả đầu ra toàn bộ quy trình này là một tệp `.jsonl`, trong đó mỗi dòng là một đối tượng JSON đại diện cho một "chunk" thông tin đã được tối ưu. Mỗi đối tượng này chứa các trường dữ liệu quan trọng:

1. `id`: Một mã định danh duy nhất được tạo bằng `uuid`.
2. `source`: Tên của văn bản gốc (ví dụ: "Chỉ thị 1722/CT-TTg").
3. `location`: Chuỗi trích dẫn đầy đủ được lấy từ `citation_path` (ví dụ: "Chương I, Mục 1, Khoản a").
4. `content`: Nội dung đã được làm giàu và tinh chỉnh kích thước.
5. Các siêu dữ liệu khác: `issue_date`, `effective_date`, `signatory`, `signatory_position`

2.2.2.6. Nhúng văn bản và lưu trữ vào ChromaDB

Sau khi đã hoàn thành việc chuyển đổi cấu trúc cây thành một danh sách các "chunk" ngữ nghĩa, bước cuối cùng trong luồng xử lý dữ liệu là chuyển đổi các "chunk" thông tin này từ dạng chữ thành một định dạng mà máy tính có thể tìm kiếm theo ngữ nghĩa, và sau đó lưu trữ chúng vào một cơ sở dữ liệu chuyên dụng. Đây là giai đoạn xây dựng nên "bộ não" có thể truy vấn được của hệ thống RAG.

Quy trình này được thiết kế để thực hiện hai nhiệm vụ cốt lõi: chuẩn bị dữ liệu dưới dạng Document và thực hiện nhúng và lập chỉ mục vào ChromaDB.

Đầu tiên, các "chunk" đã được tối ưu sẽ được đọc và chuyển đổi thành các đối tượng Document của thư viện `LangChain`. Đây là một cấu trúc dữ liệu tiêu chuẩn, đóng vai trò như một "container" chứa thông tin cho mỗi chunk. Quá trình này được cài đặt với một logic làm giàu thông tin quan trọng:

- `page_content` (Nội dung chính): Thay vì chỉ chứa nội dung thô của chunk, trường này được tạo ra bằng cách kết hợp một chuỗi văn bản được làm giàu, bao gồm các thông tin như Nguồn, Vị trí, và Nội dung. Việc đưa các siêu dữ liệu quan trọng này trực tiếp vào `page_content` nhằm mục đích cung cấp cho mô hình embedding một ngữ cảnh đầy đủ nhất có thể, từ đó tạo ra một vector biểu diễn chính xác hơn.
- `metadata` (siêu dữ liệu): Song song đó, một từ điển (dictionary) chứa các trường siêu dữ liệu riêng biệt (`id`, `source`, `location`, `issue_date`, v.v.) cũng được đính kèm. Dữ liệu có cấu trúc này sẽ không được nhúng nhưng được

lưu trữ cùng với vector, đóng vai trò cực kỳ quan trọng cho việc lọc (filtering) chính xác trong giai đoạn truy vấn sau này.

Sau khi đã có danh sách các đối tượng Document hoàn chỉnh, quy trình nhúng và lưu trữ được thực hiện.

- **Khởi tạo mô hình nhúng:** Hệ thống khởi tạo một thực thể của lớp *GoogleGenerativeAIEmbeddings*, chỉ định rõ việc sử dụng mô hình *models/text-embedding-004*. Đây là mô hình chịu trách nhiệm chuyển đổi trường *page_content* của mỗi Document thành một vector embedding.
- **Xây dựng Cơ sở dữ liệu Vector:** Toàn bộ quá trình được thực hiện thông qua một lệnh duy nhất và mạnh mẽ là *Chroma.from_documents*. Lệnh này nhận đầu vào là danh sách các đối tượng Document đã chuẩn bị, mô hình embedding đã khởi tạo, và một đường dẫn thư mục (*persist_directory*) để lưu trữ cơ sở dữ liệu. Cơ bản, lệnh đó tự động điều phối các tác vụ sau:
 1. Duyệt qua từng Document trong danh sách.
 2. Sử dụng *embedding_model* để tạo ra một vector cho *page_content* của mỗi Document.
 3. Lưu trữ mỗi vector cùng với Document gốc (bao gồm cả *page_content* và *metadata*) vào cơ sở dữ liệu ChromaDB.
 4. Xây dựng một chỉ mục (index) hiệu suất cao dựa trên các vector này để tối ưu cho việc tìm kiếm tương đồng.
 5. Lưu (persist) toàn bộ cơ sở dữ liệu đã được lập chỉ mục vào thư mục đã chỉ định trên đĩa cứng.

Kết quả của toàn bộ giai đoạn này là một cơ sở dữ liệu vector hoàn chỉnh và bền vững, được lưu trữ tại thư mục *vectorstore_path*. Đây chính là kho kiến thức đã được số hóa và lập chỉ mục, là "bộ não" mà luồng xử lý truy vấn trực tuyến sẽ tương tác để tìm kiếm thông tin và trả lời câu hỏi của người dùng.

2.2.3. Xử lý truy vấn và sinh câu trả lời

2.2.3.1. Tiền xử lý và nhúng câu hỏi

Khi người dùng nhập một câu hỏi vào giao diện ứng dụng, hệ thống sẽ tiếp nhận và chuyển đổi câu hỏi từ dạng văn bản thô sang một định dạng vector mà hệ thống có thể hiểu được về mặt ngữ nghĩa.

Quy trình này gồm hai bước nhỏ:

1. **Tiền xử lý câu hỏi:** Câu hỏi của người dùng, được tiếp nhận dưới dạng một chuỗi ký tự, sẽ được đi qua một bước tiền xử lý cơ bản. Các tác vụ chính bao gồm chuẩn hóa về chữ viết thường và loại bỏ các khoảng trắng thừa. Mục đích của bước này là để đảm bảo tính nhất quán của dữ liệu đầu vào, giúp cho việc đối chiếu và tìm kiếm ở các bước sau trở nên đáng tin cậy hơn.
2. **Nhúng vector câu hỏi:** Sau khi được chuẩn hóa, câu hỏi sẽ được đưa vào chính mô hình nhúng *text-embedding-004* của Google, vốn là mô hình đã được sử dụng để tạo vector cho kho kiến thức ở luồng xử lý dữ liệu. Việc sử

dùng cùng một mô hình là một yêu cầu bắt buộc, vì nó đảm bảo rằng vector của câu hỏi và các vector của các đoạn văn bản trong cơ sở dữ liệu nằm trong cùng một không gian ngữ nghĩa. Điều này cho phép hệ thống có thể so sánh "khoảng cách" giữa chúng một cách chính xác để tìm ra sự tương đồng về mặt ý nghĩa.

Kết quả của bước này là một vector đại diện cho ngữ nghĩa của câu hỏi người dùng, sẵn sàng được sử dụng cho bước tìm kiếm thông tin tiếp theo.

2.2.3.2. Tìm kiếm ngữ cảnh (Retrieval):

Đây là bước cốt lõi, quyết định chất lượng của các thông tin đầu vào sẽ được cung cấp cho mô hình ngôn ngữ lớn. Thay vì chỉ thực hiện một phương pháp tìm kiếm duy nhất, hệ thống được thiết kế với một **cơ chế truy xuất hybrid**, kết hợp ưu điểm của cả tìm kiếm ngữ nghĩa và tìm kiếm từ khóa để tối ưu độ chính xác.

Phương pháp hybrid này được lựa chọn vì nó mang lại sự cân bằng hoàn hảo. Vector Search rất mạnh trong việc tìm kiếm các ý nghĩa tương đồng, trong khi BM25 lại vượt trội trong việc tìm kiếm các thuật ngữ và từ khóa chính xác. Bằng cách kết hợp cả hai, hệ thống có thể vừa "hiểu" được ý định của người dùng, vừa không bỏ sót các chi tiết quan trọng, từ đó tạo ra một danh sách các ngữ cảnh đầu vào có chất lượng và độ phủ cao nhất.

Toàn bộ logic truy xuất được cài đặt trong một module retriever thông minh, hoạt động theo các bước sau:

1. **Phân tích truy vấn để lọc:** Đầu tiên, câu hỏi của người dùng sẽ được phân tích bằng các biểu thức chính quy để tìm kiếm các "gợi ý" về việc lọc dữ liệu, chẳng hạn như số hiệu văn bản ("Chỉ thị 2634/CT-VPCP") hoặc vị trí cụ thể ("Mục 1, Khoản a"). Các thông tin này sẽ được dùng để tạo ra một bộ lọc (filter), giúp thu hẹp không gian tìm kiếm chỉ trong các văn bản hoặc các phần có liên quan.
2. **Tìm kiếm ngữ nghĩa (vector search):** Hệ thống sử dụng vector của câu hỏi để thực hiện một cuộc tìm kiếm tương đồng trên cơ sở dữ liệu ChromaDB. Nó sẽ lấy ra k đoạn văn bản (chunks) có vector gần nhất với vector câu hỏi, dựa trên bộ lọc đã được tạo ở trên (nếu có).
3. **Tìm kiếm từ khóa (Keyword Search - BM25):** Để khắc phục trường hợp tìm kiếm ngữ nghĩa có thể bỏ sót các kết quả chứa từ khóa chính xác nhưng có ngữ cảnh hơi khác, hệ thống bổ sung thêm một lớp tìm kiếm từ khóa bằng thuật toán BM25. Thuật toán này sẽ được áp dụng trên tập hợp các văn bản đã được lọc theo số hiệu (nếu có), giúp tìm ra các đoạn văn bản phù hợp nhất dựa trên tần suất xuất hiện của các từ khóa trong câu hỏi.
4. **Kết hợp và loại bỏ trùng lặp:** Kết quả từ cả hai phương pháp tìm kiếm (ngữ nghĩa và từ khóa) sẽ được kết hợp lại. Một logic loại bỏ trùng lặp được áp dụng để đảm bảo mỗi đoạn văn bản chỉ xuất hiện một lần trong danh sách kết quả cuối cùng.

2.2.3.3. Sinh câu trả lời và hiển thị kết quả:

Sau khi đã có được danh sách các ngữ cảnh chất lượng cao, bước cuối cùng là tổng hợp chúng thành một câu trả lời hoàn chỉnh và hiển thị cho người dùng.

1. **Xây dựng prompt:** Các đoạn văn bản (ngữ cảnh) được truy xuất sẽ được kết hợp với câu hỏi gốc của người dùng để tạo thành một **câu lệnh (prompt)** chi tiết. Một PromptTemplate được thiết kế cẩn thận để ra chỉ thị rõ ràng cho mô hình ngôn ngữ lớn:

“Bạn là một trợ lý AI chuyên trả lời các câu hỏi về văn bản quy phạm pháp luật Việt Nam. Sử dụng những thông tin trong Ngữ cảnh dưới đây để trả lời câu hỏi của người dùng một cách chi tiết, chính xác và đầy đủ.

Hãy trả lời bằng tiếng Việt.

Luôn trích dẫn nguồn, vị trí (chương/mục/khoản), ngày ban hành, và người ký nếu có trong Ngữ cảnh.

Nếu Ngữ cảnh có nhiều phần, hãy tổng hợp thông tin liên quan một cách logic và suy luận dựa trên nội dung có sẵn.

Nếu câu hỏi kết hợp vị trí và nội dung cụ thể, hãy ưu tiên chunk khớp vị trí, sau đó suy luận từ các chunk liên quan gần nhất để trả lời đầy đủ (không thêm info ngoài Ngữ cảnh).

Nếu câu hỏi về metadata (người ký, ngày), trích dẫn từ bất kỳ chunk nào có info đó trong Ngữ cảnh, và nếu không exact, dùng chunk gần nhất.

Nếu trong Ngữ cảnh không có thông tin để trả lời, hãy nói rằng: "Tôi không tìm thấy thông tin phù hợp trong tài liệu để trả lời câu hỏi này." và gợi ý kiểm tra văn bản gốc.

Tuyệt đối không được bịa ra câu trả lời, chỉ suy luận từ Ngữ cảnh.

Ngữ cảnh:

{context}

Câu hỏi:

{question}

Câu trả lời chi tiết:”

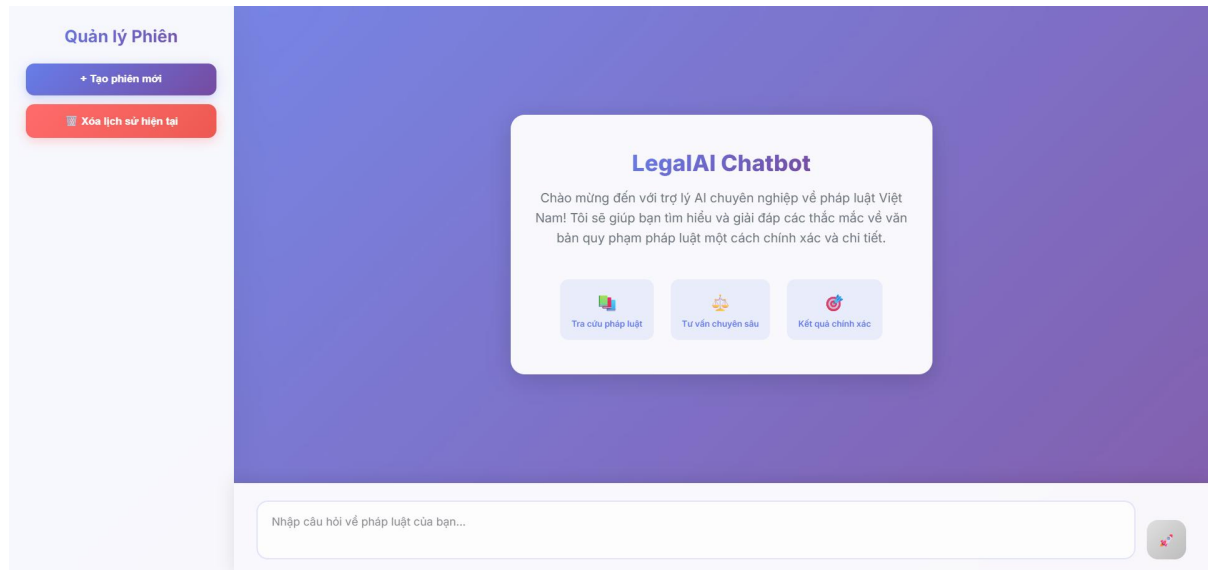
Mẫu câu lệnh này đóng vai trò cực kỳ quan trọng trong việc kiểm soát hành vi của LLM, bắt buộc nó phải trả lời dựa trên nguồn tin được cung cấp.

2. **Tạo sinh câu trả lời:** Câu lệnh hoàn chỉnh sau đó được gửi đến mô hình Gemini 2.5 Pro thông qua API. Mô hình sẽ đọc và "suy luận" trên ngữ cảnh đã cho để tạo ra một câu trả lời bằng ngôn ngữ tự nhiên, mạch lạc và bám sát vào các quy định pháp luật.
3. **Hiển thị kết quả:** Câu trả lời do Gemini tạo ra sẽ được gửi trở lại backend của ứng dụng **Flask**. Sau đó, thông qua API endpoint /query, câu trả lời được truyền đến giao diện người dùng và hiển thị trong khu vực hội thoại. Toàn bộ logic tương tác ở phía người dùng (gửi câu hỏi, nhận và hiển thị câu trả lời) được quản lý bởi mã JavaScript, tạo ra một trải nghiệm chat mượt mà và tức thời.

CHƯƠNG 3. KIỂM THỬ VÀ ĐÁNH GIÁ

3.1. Giao diện sản phẩm

Để người dùng có thể tương tác với hệ thống một cách hiệu quả, một giao diện web hoàn chỉnh đã được xây dựng bằng framework Flask cho phần backend và HTML, CSS, JavaScript cho phần frontend. Giao diện được thiết kế theo phong cách hiện tại, tối giản và chuyên nghiệp, tập trung vào việc mang lại trải nghiệm hỏi-đáp mượt mà và trực quan cho người dùng.



Hình 18. Giao diện hệ thống

Như trong hình 18, giao diện chính của ứng dụng được chia thành hai khu vực: Thanh điều khiển bên trái và khu vực hội thoại chính bên phải.

3.1.1. Thanh điều khiển - Quản lý phiên (Sidebar):

Thanh điều khiển bên trái đóng vai trò là trung tâm quản lý các phiên làm việc của người dùng, giúp họ tổ chức các cuộc hội thoại một cách khoa học. Các chức năng chính bao gồm:

- **Tạo phiên mới:** Nút "+ Tạo phiên mới" cho phép người dùng bắt đầu một cuộc hội thoại hoàn toàn mới. Điều này rất hữu ích khi người dùng muốn chuyển sang một chủ đề tra cứu khác mà không làm ảnh hưởng đến lịch sử của các chủ đề đang dang dở.
- **Xóa lịch sử hiện tại:** Nút này dùng để xóa toàn bộ nội dung trong cuộc hội thoại hiện tại, giúp làm sạch không gian làm việc khi cần thiết.
- **Danh sách các phiên đã lưu:** Bên dưới các nút chức năng, hệ thống sẽ hiển thị một danh sách các phiên hội thoại đã được lưu trước đó. Người dùng có thể dễ dàng nhấp vào để chuyển đổi qua lại giữa các phiên, cũng như thực hiện các thao tác đổi tên hoặc xóa từng phiên cụ thể.

3.1.2. Khu vực hội thoại chính (Main Chat Area):

Đây là không gian tương tác chính giữa người dùng và trợ lý ảo.

- **Màn hình chào mừng:** Khi chưa có phiên hội thoại nào được bắt đầu, một màn hình chào mừng sẽ xuất hiện, giới thiệu ngắn gọn về khả năng của chatbot và gợi ý các chức năng chính như "Tra cứu pháp luật", "Kết quả chính xác".
- **Hiển thị hội thoại:** Khi một phiên chat diễn ra, các tin nhắn sẽ được hiển thị theo định dạng hỏi-đáp quen thuộc, với câu hỏi của người dùng và câu trả lời của trợ lý ảo được phân biệt rõ ràng về màu sắc và vị trí.
- **Khung nhập liệu:** Phía dưới cùng là một ô nhập liệu lớn, cho phép người dùng gõ các câu hỏi dài và phức tạp một cách thoải mái, đi kèm với một nút gửi trực quan.

3.2. Kết quả thực nghiệm

3.2.1. Thống kê dữ liệu qua các bước tiền xử lý:

Trước khi đi vào đánh giá hiệu suất của mô hình hỏi-đáp, phần này trình bày các kết quả và thống kê định lượng của dữ liệu qua từng giai đoạn trong pipeline tiền xử lý. Việc này giúp cung cấp một cái nhìn tổng quan về đặc điểm của bộ dữ liệu đã được xây dựng.

Quá trình tiền xử lý dữ liệu bắt đầu với **37 tệp văn bản PDF** thuộc loại "Chỉ thị" được thu thập từ Cổng Thông tin điện tử Chính phủ. Sau khi đi qua các bước trích xuất, làm sạch và bóc tách cấu trúc, 48 tệp PDF này đã được chuyển đổi thành công thành **37 đối tượng văn bản có cấu trúc** trong tệp `processed_documents_chithi.json`.

Từ 37 văn bản có cấu trúc này, thuật toán duyệt cây đệ quy đã được áp dụng để tạo ra các chunk ngữ nghĩa ban đầu. Kết quả của giai đoạn này là **716 chunk ngữ nghĩa thô**. Mỗi chunk này đều được làm giàu với các thông tin ngữ cảnh từ các cấp cha (Chương, Mục,...) và các siêu dữ liệu quan trọng.

Cuối cùng, 716 chunk thô này được đưa qua quy trình tinh chỉnh kích thước hybrid. Các chunk quá dài sẽ được chia nhỏ bằng `RecursiveCharacterTextSplitter` và `SemanticChunker`. Quy trình này đã làm tăng tổng số chunk lên **731 chunk cuối cùng**, sẵn sàng cho việc nhúng và lưu trữ. Sự gia tăng này cho thấy một số chunk ban đầu có nội dung dài đã được chia nhỏ thành các đơn vị hợp lý hơn, trong khi vẫn đảm bảo tính toàn vẹn về mặt ngữ nghĩa.

Bảng 2. Bảng thống kê quá trình xử lý dữ liệu

Giai đoạn	Đơn vị	Số lượng
Thu thập	Tệp PDF	37
Tái cấu trúc	Văn bản có cấu trúc (JSON)	37
Tạo chunk ban đầu	Chunk ngữ nghĩa thô	716
Tinh chỉnh chunk	Chunk cuối cùng (sẵn sàng để nhúng)	731

3.2.2. Thiết lập môi trường và dữ liệu thực nghiệm:

Môi trường thực nghiệm:

- **Phần cứng:** Toàn bộ quá trình nhúng và kiểm thử được thực hiện trên nền tảng Google Colab, sử dụng GPU T4.
- **Phần mềm:** Hệ thống được xây dựng trên ngôn ngữ Python 3.12, với backend được triển khai bằng Flask 3.1.1. Các thư viện AI/ML chính bao gồm LangChain 0.3.27, , chromaDB 1.0.17, và các thư viện trong hệ sinh thái Google Generative AI.

Bộ dữ liệu đánh giá: Bộ dữ liệu bao gồm 35 cặp câu hỏi và câu trả lời tham chiếu (ground_truth).

- **Câu hỏi (questions):** 35 câu hỏi được thiết kế để mô phỏng các truy vấn thực tế của người dùng, bao phủ nhiều loại văn bản và các khía cạnh thông tin khác nhau.
- **Câu trả lời tham chiếu (ground_truths):** Với mỗi câu hỏi, một câu trả lời chính xác đã được soạn thảo thủ công dựa trên việc tra cứu trực tiếp từ các văn bản gốc. Các câu trả lời này đóng vai trò làm "đáp án vàng" để so sánh và tính toán độ chính xác ngữ nghĩa của câu trả lời do hệ thống tạo ra.

Các câu hỏi trong bộ dữ liệu được thiết kế một cách có chủ đích để kiểm tra các năng lực khác nhau của hệ thống RAG, từ đơn giản đến phức tạp. Chúng được phân thành các loại sau:

- **Loại 1: Câu hỏi tra cứu siêu dữ liệu:** Đây là các câu hỏi đơn giản, yêu cầu hệ thống trích xuất các thông tin định danh cơ bản của văn bản như người ký, ngày ban hành, hoặc số hiệu. Loại câu hỏi này dùng để kiểm tra khả năng hệ thống truy xuất và đọc các thông tin đã được bóc tách ở bước tiền xử lý. *Ví dụ:* "Ai là người ký Chỉ thị số 1722/CT-TTg và chức vụ của người đó là gì?"*
- **Loại 2: Câu hỏi trích xuất thông tin trực tiếp:** Đây là loại câu hỏi phổ biến nhất, yêu cầu hệ thống tìm đến một quy định cụ thể và trích xuất một thông tin, số liệu chính xác nằm trong đó. Loại câu hỏi này đánh giá hiệu quả của toàn bộ pipeline, từ việc tìm đúng "chunk" văn bản cho đến việc LLM đọc hiểu và rút ra câu trả lời đúng. *Ví dụ:* "Theo Chỉ thị 1752/CT-TTg, mức chuẩn nghèo áp dụng cho khu vực nông thôn giai đoạn 2011 - 2015 là bao nhiêu?"*
- **Loại 3: Câu hỏi có điều kiện lọc:** Các câu hỏi này chứa các từ khóa chỉ định rõ một văn bản hoặc một vị trí cụ thể (Chương, Mục, Khoản). Chúng được thiết kế để đánh giá trực tiếp hiệu quả của **cơ chế truy xuất hybrid**, kiểm tra xem module có khả năng thu hẹp không gian tìm kiếm dựa trên các bộ lọc được phân tích từ câu hỏi hay không. *Ví dụ:* "Theo Khoản b, Mục 1, Chỉ thị 07/CT-TTg, Thủ tướng yêu cầu tạm dừng việc gì liên quan đến khu kinh tế (KKT), khu công nghiệp (KCN)?"*

Bảng 3. Phân loại và ví dụ các câu hỏi trong bộ dữ liệu đánh giá

Loại câu hỏi	Ví dụ	Mục tiêu đánh giá
Tra cứu siêu dữ liệu	Ai là người ký Chỉ thị 1685/CT-TTg?	Khả năng trích xuất thông tin định danh.
Trích xuất trực tiếp	Mức chuẩn nghèo khu vực nông thôn theo Chỉ thị 1752/CT-TTg là bao nhiêu?	Khả năng tìm và đọc đúng một quy định cụ thể.
Truy vấn có lọc	Theo Khoản b, Mục 1, Chỉ thị 07/CT-TTg, Thủ tướng yêu cầu tạm dừng việc gì?	Hiệu quả của module truy xuất hybrid (lọc và tìm kiếm).

Bộ dữ liệu được thiết kế đa dạng này sẽ là cơ sở vững chắc để thực hiện các bước đánh giá định lượng và định tính một cách toàn diện ở các mục tiếp theo.

3.2.3. Kết quả đánh giá định lượng:

Để có một góc nhìn khách quan và dựa trên số liệu về hiệu suất của hệ thống, bộ dữ liệu gồm 30 câu hỏi đã được sử dụng để đánh giá thông qua framework **RAGAS**. Quá trình này giúp đo lường các khía cạnh khác nhau của pipeline RAG, từ chất lượng của module truy xuất cho đến độ tin cậy của câu trả lời do LLM tạo ra.

Kết quả đánh giá định lượng được tổng hợp trong bảng dưới đây:

Bảng 4. Kết quả đánh giá hiệu suất hệ thống bằng RAGAS

Chỉ số	Điểm số	Phân tích và diễn giải
Faithfulness	0.9833	Điểm số gần như tuyệt đối này cho thấy hệ thống có tính trung thực rất cao . Câu trả lời do mô hình Gemini tạo ra hầu như luôn bám sát và có thể được suy ra hoàn toàn từ các đoạn văn bản ngữ cảnh được cung cấp. Điều này khẳng định hệ thống đã giải quyết rất tốt vấn đề "ảo giác" (hallucination).
Context Precision	1.0000	Điểm số hoàn hảo cho thấy module truy xuất hoạt động với độ chính xác tuyệt đối . Toàn bộ các đoạn ngữ cảnh mà hệ thống tìm thấy đều thực sự liên quan đến câu hỏi của người dùng, không có thông tin "nhiều". Điều này tối ưu hóa đầu vào cho LLM, giúp nó tập trung vào đúng thông tin cần thiết.
Context Recall	0.9545	Điểm số rất cao này cho thấy module truy xuất không chỉ chính xác mà còn có độ phủ tốt . Hệ thống đã thành công trong việc tìm thấy gần như toàn bộ các thông tin cần thiết nằm rải rác trong cơ sở dữ liệu để có thể đưa ra một câu trả lời đầy đủ.
Answer Relevancy	NaN	Chỉ số này đo lường mức độ câu trả lời đi thẳng vào trọng tâm câu hỏi. Do các giới hạn về tài nguyên và thời gian trong quá trình thực nghiệm như API timeout, một điểm số hoàn chỉnh cho chỉ số này đã không thể được thu thập. Tuy nhiên, độ liên quan của câu trả lời có thể được phân tích một cách rõ ràng thông qua các ví dụ định tính ở mục tiếp theo.

3.2.4. Kết quả đánh giá định tính:

Bên cạnh các số liệu định lượng, việc phân tích các trường hợp cụ thể sẽ cung cấp một cái nhìn sâu sắc hơn về khả năng thực tế của hệ thống. Dưới đây là một số ví dụ tiêu biểu.

Trường hợp 1: Tra cứu siêu dữ liệu

- **Mục tiêu đánh giá:** Kiểm tra khả năng trích xuất các thông tin định danh cơ bản của văn bản.
- **Câu hỏi:** Ai là người ký Chỉ thị 1685/CT-TTg?
- **Kết quả:**

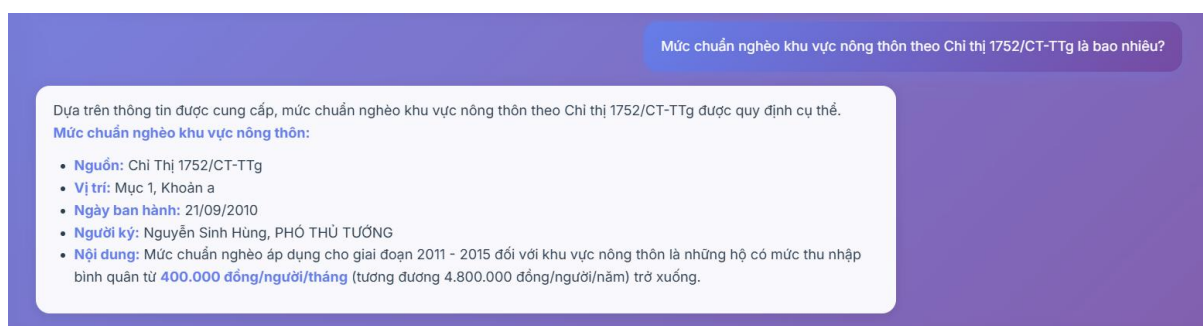


Hình 19. Kết quả trường hợp 1

- **Kết luận:** Hệ thống đã trả lời hoàn toàn chính xác. Câu trả lời ngắn gọn, đi thẳng vào vấn đề và trích dẫn đầy đủ các thông tin liên quan. Điều này cho thấy bước tiền xử lý đã bóc tách và làm giàu siêu dữ liệu vào các chunk một cách hiệu quả, và module retriever có thể dễ dàng truy xuất được các thông tin này.

Trường hợp 2: Trích xuất thông tin trực tiếp

- **Mục tiêu đánh giá:** Kiểm tra khả năng tìm và đọc đúng một quy định cụ thể trong văn bản.
- **Câu hỏi:** Mức chuẩn nghèo khu vực nông thôn theo Chỉ thị 1752/CT-TTg là bao nhiêu?
- **Kết quả:**



Hình 20. Kết quả trường hợp 2

- **Kết luận:** Câu trả lời này thể hiện sự thành công của toàn bộ pipeline. Hệ thống đã xác định đúng văn bản (1752/CT-TTg), tìm kiếm đúng cụm từ khóa ("mức

chuẩn nghèo khu vực nông thôn"), truy xuất được đoạn ngữ cảnh chính xác, và cuối cùng là LLM đã đọc hiểu và rút ra con số cụ thể.

Trường hợp 3: Truy vấn có Lọc

- **Mục tiêu đánh giá:** Kiểm tra hiệu quả của module truy xuất hybrid (lọc và tìm kiếm) khi xử lý các câu hỏi phức tạp, có chỉ định rõ vị trí.
- **Câu hỏi:** Theo Khoản b, Mục 1, Chỉ thị 07/CT-TTg, Thủ tướng yêu cầu tạm dừng việc gì?
- **Kết quả:**

The screenshot shows a search interface with a purple header. A search bar at the top right contains the text: "Theo Khoản b, Mục 1, Chỉ thị 07/CT-TTg, Thủ tướng yêu cầu tạm dừng việc gì?". Below the search bar, a white box displays the search results. The text in the box reads: "Dựa trên thông tin được cung cấp, theo Khoản b, Mục 1 của Chỉ thị 07/CT-TTg, Thủ tướng yêu cầu tạm dừng việc bổ sung quy hoạch và thành lập mới các Khu kinh tế (KKT), Khu công nghiệp (KCN) trên cả nước." Below this, under the heading "Chi tiết:", there is a list of details: "Văn bản: Chỉ thị 07/CT-TTg", "Ngày ban hành: 02/03/2012", "Người ký: Nguyễn Tấn Dũng, THỦ TƯỚNG", and "Vị trí: Mục 1, Khoản b". Under the heading "Nội dung yêu cầu tạm dừng:", it states: "Trong quá trình thực hiện rà soát điều chỉnh Quy hoạch phát triển KKT, KCN Việt Nam đến năm 2020, Thủ tướng yêu cầu:" followed by a list of requirements: "Tạm dừng việc bổ sung quy hoạch các KKT, KCN.", "Tạm dừng việc thành lập mới các KKT, KCN trên cả nước.", and "Việc tạm dừng này sẽ kéo dài cho đến khi có chỉ đạo mới của Thủ tướng Chính phủ." At the bottom, the source is cited: "Nguồn: Chỉ thị 07/CT-TTg, Mục 1, Khoản b, ban hành ngày 02/03/2012, do Thủ tướng Nguyễn Tấn Dũng ký."

Hình 21. Kết quả trường hợp 3

- **Kết luận:** Đây là một trường hợp phức tạp, đòi hỏi hệ thống phải thực hiện nhiều bước. Đầu tiên, module phân tích truy vấn đã nhận diện được các bộ lọc (source='Chỉ thị 07/CT-TTg' và location='Mục 1, Khoản b'). Các bộ lọc này giúp thu hẹp không gian tìm kiếm một cách đáng kể. Sau đó, module retriever đã ưu tiên và lấy ra chính xác chunk văn bản tương ứng với vị trí này. Kết quả là một câu trả lời cực kỳ chính xác và liên quan. Điều này chứng tỏ thiết kế retriever hybrid là rất hiệu quả trong thực tế.

III. PHẦN KẾT LUẬN

1. Kết quả đạt được

Luận văn đã thực hiện thành công mục tiêu đề ra là xây dựng một hệ thống trợ lý ảo hoàn chỉnh, có khả năng hỗ trợ hiệu quả hoạt động tra cứu và hỏi-đáp trên các văn bản quy phạm pháp luật tiếng Việt. Xuyên suốt quá trình thực hiện, đề tài đã đạt được những kết quả cụ thể và có giá trị. Thứ nhất, một pipeline tiền xử lý dữ liệu chi tiết đã được xây dựng, có khả năng tự động hóa việc trích xuất, làm sạch, và đặc biệt là tái tạo lại cấu trúc logic của văn bản pháp luật từ các tệp PDF. Phương pháp phân đoạn theo ngữ nghĩa (semantic chunking) đã chứng tỏ hiệu quả vượt trội so với các cách tiếp cận cơ học, tạo ra các đơn vị thông tin (chunk) giàu ngữ cảnh và chất lượng cao, đây là tiền đề quan trọng cho sự thành công của toàn bộ hệ thống.

Thứ hai, một kiến trúc RAG (Retrieval-Augmented Generation) tiên tiến đã được cài đặt thành công, kết hợp sức mạnh của các mô hình AI hàng đầu. Hệ thống đã triển khai một cơ chế truy xuất hybrid, kết hợp tìm kiếm ngữ nghĩa bằng vector và tìm kiếm từ khóa BM25, giúp tối ưu hóa khả năng tìm kiếm chính xác cả về ý nghĩa và thuật ngữ. Việc tích hợp mô hình ngôn ngữ lớn Gemini 2.5 Pro đã cho phép hệ thống không chỉ trích xuất mà còn có thể tổng hợp, suy luận và diễn đạt lại thông tin một cách tự nhiên, mạch lạc. Kết quả kiểm thử định lượng thông qua framework RAGAS đã chứng minh hiệu suất cao của hệ thống với các chỉ số Faithfulness (0.9833) và Context Precision (1.0000) gần như tuyệt đối, khẳng định khả năng trả lời chính xác và đáng tin cậy, giải quyết tốt bài toán "ảo giác". Cuối cùng, một ứng dụng web hoàn chỉnh sử dụng Flask đã được xây dựng, cung cấp một giao diện người dùng chuyên nghiệp và trực quan, biến mô hình lý thuyết thành một sản phẩm có tính ứng dụng thực tiễn.

2. Hướng phát triển

Bên cạnh những kết quả đã đạt được, luận văn cũng mở ra nhiều hướng phát triển tiềm năng trong tương lai để tiếp tục hoàn thiện và nâng cao năng lực của hệ thống.

Đầu tiên, hướng phát triển quan trọng nhất là **mở rộng và đa dạng hóa kho kiến thức**. Hệ thống hiện tại mới chỉ được huấn luyện và kiểm thử trên các văn bản thuộc loại "Chỉ thị". Việc bổ sung thêm các loại văn bản quy phạm pháp luật khác (như Nghị định, Luật, Thông tư) và các văn bản của chính quyền địa phương (ví dụ: các quyết định, nghị quyết của tỉnh An Giang) sẽ làm tăng đáng kể phạm vi áp dụng và tính hữu ích của trợ lý ảo. Cùng với đó, việc tích hợp khả năng xử lý các tệp PDF dạng ảnh (image-based PDF) bằng công nghệ nhận dạng ký tự quang học (OCR) sẽ giúp hệ thống xử lý được một nguồn tài liệu lớn hơn trong thực tế.

Thứ hai, **cải tiến module truy xuất thông tin** là một hướng đi đầy hứa hẹn. Mặc dù cơ chế hybrid hiện tại hoạt động hiệu quả, việc triển khai một module **Tái xếp hạng (Reranker)** chuyên dụng sau bước truy xuất có thể giúp hệ thống chọn lọc các ngữ cảnh phù hợp nhất một cách tinh vi hơn nữa, từ đó nâng cao chất lượng đầu vào cho LLM và cải thiện độ chính xác của câu trả lời cuối cùng.

Cuối cùng, về mặt tương tác người dùng và tối ưu mô hình, hệ thống có thể được **bổ sung các tính năng nâng cao**. Việc thêm cơ chế thu thập phản hồi của người dùng sẽ tạo ra một bộ dữ liệu quý giá cho việc tinh chỉnh (fine-tuning) mô hình trong tương lai. Hơn nữa,

việc nghiên cứu và áp dụng các kỹ thuật quản lý bộ nhớ hội thoại tiên tiến sẽ giúp trợ lý ảo có khả năng xử lý các chuỗi câu hỏi nối tiếp phức tạp hơn, mang lại một trải nghiệm đối thoại thực sự thông minh và tự nhiên.

TÀI LIỆU THAM KHẢO

- [1] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). *Attention is All You Need*. arXiv preprint arXiv:1706.03762.
- [2] Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, Douwe Kiela, (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. arXiv preprint:2005.11401
- [3] CTU-LinguTechies (2023) *VN-Law-Advisor: Hệ thống hỏi đáp pháp luật dựa trên mô hình ngôn ngữ lớn*.
- [4] Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2018). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. arXiv preprint arXiv:1810.04805.
- [5] Es, S., Ramamurthy, R., Khadivi, S., de Boer, M., & Versteegh, M. (2023). *RAGAS: Automated Evaluation of Retrieval Augmented Generation*. arXiv preprint arXiv:2309.15217.
- [6] Nguyen, D. Q., & Nguyen, A. T. (2020). *PhoBERT: Pre-trained language models for Vietnamese*. In Findings of the Association for Computational Linguistics: EMNLP 2020.
- [7] Chen, D., Fisch, A., Weston, J., & Bordes, A. (2017). *Reading Wikipedia to Answer Open-Domain Questions*. arXiv preprint arXiv:1704.00051.
- [8] LangChain. (2024). *LangChain Documentation*. Retrieved from <https://www.langchain.com/>.
- [9] Chroma. (2024). *ChromaDB Documentation*. Retrieved from <https://www.trychroma.com/>

